Informe de software de la Tarea 1

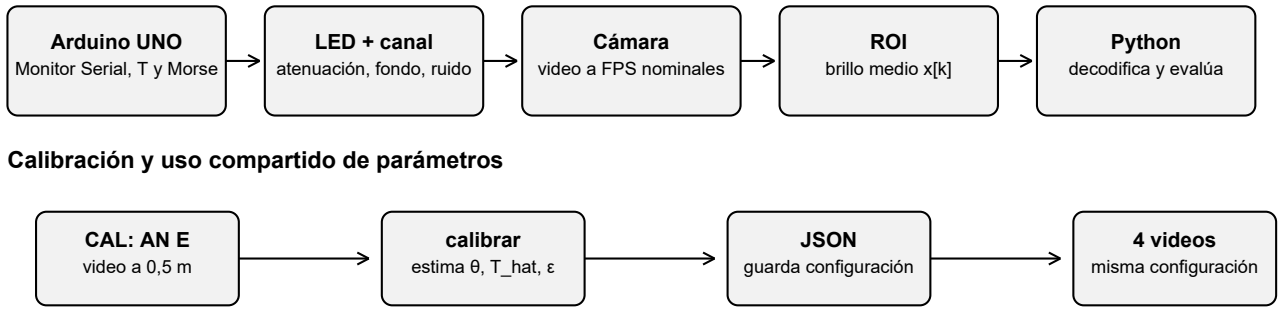
Sistemas de Comunicaciones · guía de lectura del código · 15 de octubre de 2026

Este informe describe la implementación existente, no resultados experimentales del grupo. Explica los **cuatro archivos Python** (`ejecutar_tarea1.py`, `siscom/receptor.py`, `siscom/analisis.py` y `siscom/procesar.py`) y, para cubrir el enlace completo, el transmisor Arduino `transmisor_morse.ino`. El Arduino recibe las órdenes exclusivamente desde el Monitor Serial del Arduino IDE; Python no controla el puerto. El receptor reconstruye el mensaje desde el video antes de compararlo con `SIS COM`.

1. Mapa completo del sistema

El flujo físico es: texto en Monitor Serial -> Arduino UNO -> LED -> espacio libre e iluminación ambiente -> cámara -> video -> brillo promedio de una región de interés (ROI) -> decisión ON/OFF -> duraciones Morse -> texto recibido. Después de esa reconstrucción vienen CER, FFT, PSD, B95 y gráficos. La calibración usa otro video, grabado a 0,5 m, con la secuencia conocida `AN E`.

Figura 1. Cadena completa de transmisión y recepción



Hay cuatro **ensayos** de `SIS COM`: uno a 0,5 m y otros a 1, 2 y 3 m. `CAL` es una grabación adicional, no un quinto ensayo de evaluación. La palabra nueva de cuatro letras es una demostración posterior con la misma configuración de recepción.

Archivo	Responsabilidad	Lo ejecuta el estudiante
`transmisor_morse.ino`	Interpreta `T`, `CAL` y `SEND`, y conmuta el LED.	Sí, desde Arduino IDE y Monitor Serial.
`ejecutar_tarea1.py`	Define los comandos `patron`, `calibrar`, `decodificar` y `evaluar`.	Sí, desde PowerShell.
`siscom/receptor.py`	Patrón Morse ideal, lectura del video, ROI, calibración y decodificación.	No; lo importa el programa principal.
`siscom/analisis.py`	FFT, PSD, B95, CER y figuras.	No; lo importa el procesamiento.
`siscom/procesar.py`	Encadena las etapas y exporta cada ensayo y la comparación.	No; lo importa el programa principal.

Dependencias concretas

El `.ino` usa las funciones de Arduino (`Serial`, `digitalWrite`, `micros`) y la biblioteca estándar C para comparar cadenas. Python usa `argparse`, `pathlib`, `json`, `csv` y bibliotecas científicas: OpenCV (`cv2`) para leer cuadros, NumPy para vectores y FFT, SciPy para Welch y Matplotlib para gráficos. `requirements.txt` fija las versiones de estas cuatro dependencias externas. No se usa `pyserial`, MATLAB ni la interfaz web en este flujo.

2. Variables y criterios matemáticos del software

2.1 Señal transmitida y canal

En *Modelos análogos de Canales de Comunicaciones* se nombran `s(t)` como señal de entrada al canal, `n(t)` como ruido aditivo y `r(t)` como señal de salida que entra al receptor. El modelo introductorio es `r(t) = s(t) + n(t)`. Para este montaje conviene afinarlo como `r_d(t) = a_d s(t) + b_d(t) + n_d(t)`: `a_d` representa la atenuación con la distancia `d`, `b_d(t)` la luz de fondo y `n_d(t)` las perturbaciones de cámara y ambiente. Es una **adaptación explicativa**, no una ecuación medida por el programa. El código no estima `a_d`, SNR ni una función de transferencia del canal.

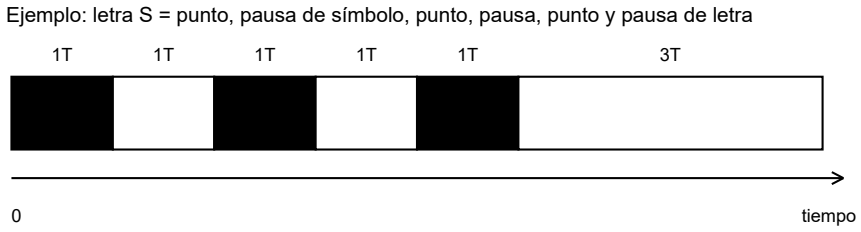
El Arduino construye una envolvente binaria `m(t)  {0,1}`: LED encendido cuando `m(t)=1`, apagado cuando `m(t)=0`. Es modulación óptica de intensidad tipo OOK. Aquí no se genera una portadora sinusoidal de AM: la información se representa por presencia o ausencia de luz visible durante intervalos Morse. Los apuntes de canales y espectro permiten razonar sobre atenuación, ruido y análisis frecuencial, pero no implican que el trayecto cámara–LED sea un SLTI ideal.

2.2 Unidad temporal T y código Morse

El tiempo base del transmisor se llama `T`. Un punto dura `T`, una raya `3T`, una separación interior de letra `T`, entre letras `3T` y entre palabras `7T`. El programa añade `7T` apagados antes y después de cada transmisión para que el video tenga margen. Esos márgenes **no** forman parte del patrón Morse del texto.

`SIS COM` se codifica `... / .. / ... // -.-. / --- / --`. Su patrón interno suma `61T`; con `T = 0,300 s`, el mensaje ocupa `18,3 s`. Los dos márgenes agregan `14T`, por lo que la secuencia luminosa completa comandada ocupa aproximadamente `75T = 22,5 s`, aparte del pequeño tiempo de preparación e impresión serial. La calibración `AN E` incluye puntos, rayas, separaciones de símbolo, letra y palabra; así permite estimar el umbral y `T` con una señal conocida.

Figura 2. Unidades de tiempo Morse usadas por el transmisor



Negro: LED ON. Blanco: LED OFF. Entre palabras el silencio dura 7T.

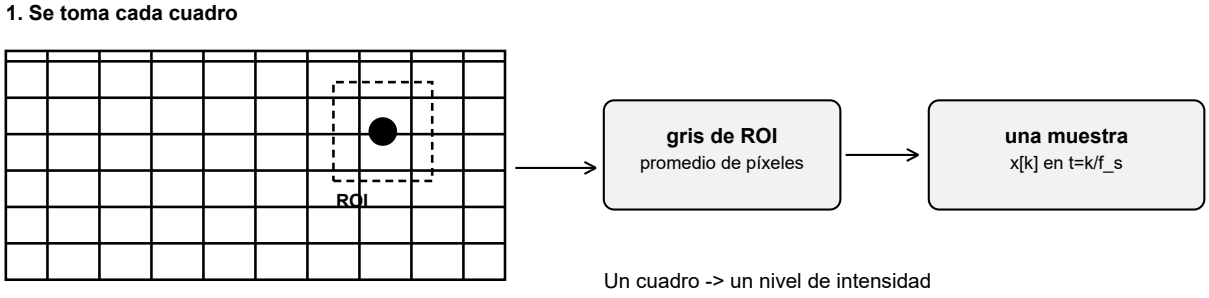
2.3 Cámara, muestreo y ROI

Los apuntes *Nyquist* y *BW* usan `x(t)` para la señal continua, `X(f)` para su espectro, `T_0` para el período de muestreo y `f_0 = 1/T_0` para la frecuencia de muestreo. Aquí se toma `f_s = FPS` y `T_s = 1/f_s`. Cada cuadro entrega una muestra de intensidad media de la ROI:

`x[k] = (1/|ROI|) Σ_(p ROI) gris(cuadro_k,p)`, con tiempo aproximado `t_k = k/f_s`.

Esta `x[k]` es brillo digital de cámara, no potencia óptica calibrada en watts. Se usa la misma ROI en la secuencia seleccionada; en los ensayos se conservan ancho y alto calibrados. La cámara informa sus FPS nominales y el código contrasta, cuando el backend los entrega de manera fiable, los timestamps de cuadros con `1/f_s`.

Figura 3. Del cuadro de cámara a la señal x[k]

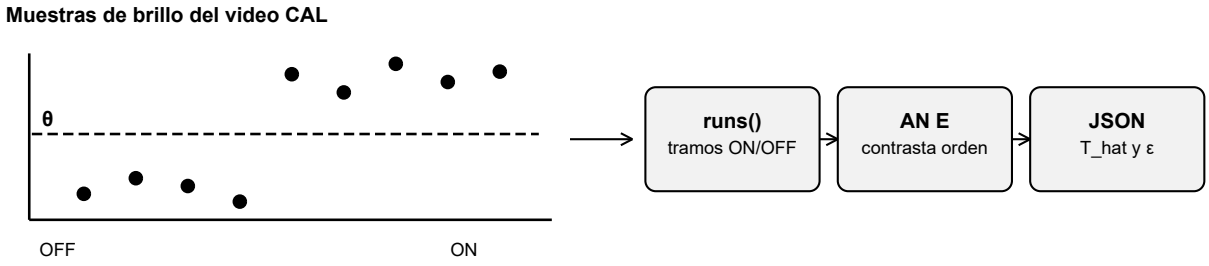


El material de Nyquist plantea ausencia de solapamiento espectral cuando `f_0 > 2f_m` para una señal limitada a `f_m`. Un pulso rectangular Morse ideal tiene transiciones abruptas y no es estrictamente limitado en banda, de modo que esa condición no basta para garantizar decodificación perfecta. La exigencia práctica del receptor es disponer de **varios cuadros por T**: la resolución temporal es `T_s` y un borde puede desplazar la duración medida en aproximadamente un cuadro. Por ejemplo, a 30 FPS y `T=0,300 s` hay unas 9 muestras por punto; a 60 FPS, unas 18. Es una estimación de diseño, no una garantía de CER cero.

2.4 Umbral, duraciones y decisión

La calibración estima los niveles `L_OFF` y `L_ON` de la señal `x[k]`. Define `θ = (L_OFF + L_ON)/2` y decide `m_hat[k] = 1` si `x[k] > θ`, y `0` en caso contrario. Al agrupar muestras consecutivas del mismo estado se obtiene una duración observada `Δt_j`; su número de unidades es `u_j = Δt_j / T_hat`. El decodificador compara `u_j` con los valores permitidos: `1` o `3` cuando el LED está ON y `1`, `3` o `7` cuando está OFF. Solo acepta la clase si `|u_j - u_ideal| ≤ ε`, donde `ε` está expresada en unidades de `T`.

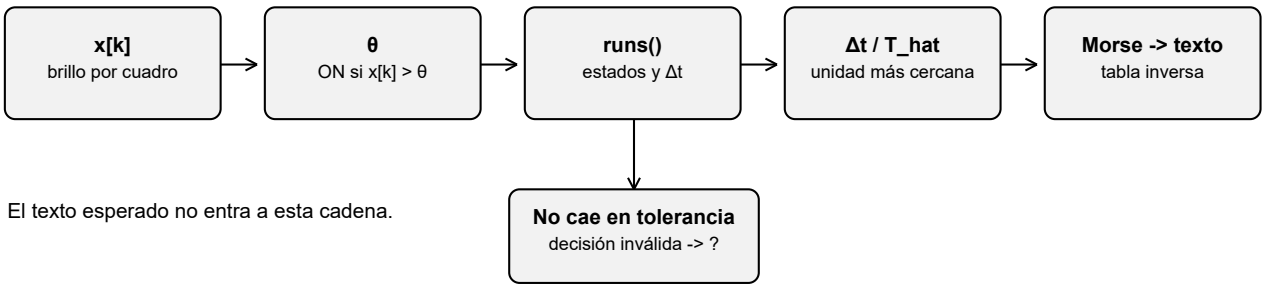
Figura 4. Cómo la secuencia CAL produce los parámetros



Calibrate() guarda el umbral medio entre niveles y estima T con la mediana de duración/unidades.

La decisión es temporal y por umbral, no una correlación con la palabra que se espera recibir. Un pulso inválido añade `!` al patrón, que termina como `?`; una pausa inválida también se registra como error. Esto impide que el receptor “corrija” silenciosamente una recepción mala hacia `SIS COM`.

Figura 5. Decisión temporal del decodificador

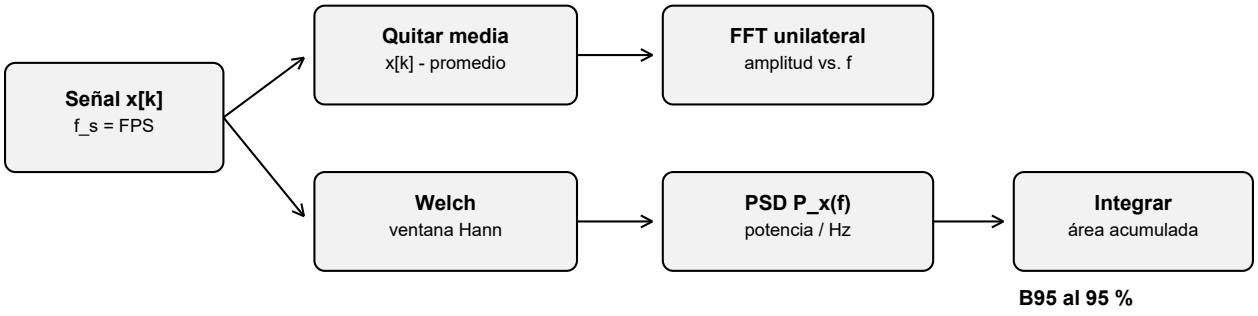


2.5 Espectro, PSD y ancho B95

Los apuntes *Series y transformadas de Fourier* trabajan con `x(t)` y `X(f)`; *Respuesta espectral SLTI* relaciona la densidad espectral de potencia `P_x(f)` con el área que representa potencia. El programa primero retira la media: `x_0[k] = x[k] - promedio(x)`. Luego calcula la FFT unilateral `X[q]` y la amplitud `|X[q]|/N` (duplicando las frecuencias interiores para señal real). La abscisa es `f_q = q f_s/N`, desde 0 hasta `f_s/2`.

La PSD se **estima** con Welch, ventana Hann, segmentos de hasta 256 muestras y solapamiento de 50 %. Se integra numéricamente por trapecios. `B95` es la frecuencia que deja bajo ella 95 % del área de esta PSD unilateral: `∫_0^B95 P_x(f)df = 0,95 ∫_0^(f_s/2) P_x(f)df`. Aquí B95 es un **ancho efectivo de la señal de brillo observada**. No debe confundirse con el BW de 3 dB de un canal físico ni con el ancho de banda de la luz visible. La FFT y la PSD describen la secuencia grabada, incluido ruido y efectos de la cámara.

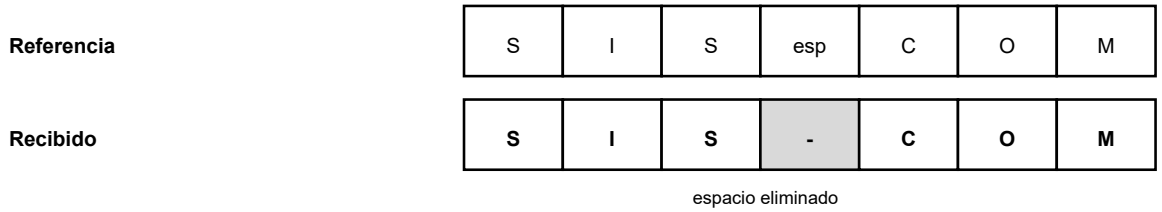
Figura 6. Ramas FFT y PSD del análisis espectral



2.6 Error de caracteres

La palabra transmitida conocida solo entra al cálculo de evaluación. Si `N_ref = 7` caracteres para `SIS COM` (seis letras y un espacio), el error es `CER = (S + D + I)/N_ref`, donde `S`, `D` e `I` son sustituciones, eliminaciones e inserciones de una alineación de Levenshtein. Por ejemplo, recibir `SISCOM` elimina el espacio: `D=1`, `CER=1/7`. Un signo `?` también cuenta como carácter incorrecto. Esta métrica se calcula después de decodificar; no influye en la decisión ON/OFF.

Figura 7. Ejemplo de alineación para el cálculo de CER



S=0, D=1, I=0 -> errores=1 -> CER=1/7

3. Transmisor Arduino: `transmisor_morse.ino`

Bloque A — constantes, buffers y tabla Morse (líneas 1–27)

`PIN_LED=8` selecciona el pin que maneja el LED externo. `SINCRONIZAR_LED_INTEGRADO` hace que el LED incorporado siga el mismo estado, pero el montaje se observa en D8. `duracionUnidadMs` arranca en 300 ms. Los buffers de tamaño fijo guardan la orden serial y el texto normalizado: el UNO evita `String` y asignación dinámica. `CODIGO_MORSE[26]` guarda las letras A–Z en orden alfabético; el índice de una letra es `letra - 'A'`. El espacio se resuelve como pausa entre palabras, no como entrada de esa tabla.

Bloque B — `esEspacio`, `setLed` y `segment` (líneas 32–53)

`esEspacio` reconoce separadores en las órdenes recibidas. `setLed` fija D8 en HIGH/LOW y, si corresponde, el LED integrado. `segment` suma las unidades `T`; cuando `emitir=true`, cambia el estado y espera hasta un plazo calculado con `micros()`. Los plazos acumulados evitan sumar el tiempo que tarda `digitalWrite` en cada tramo. Durante la secuencia no imprime por Serial, para no alterar innecesariamente la temporización.

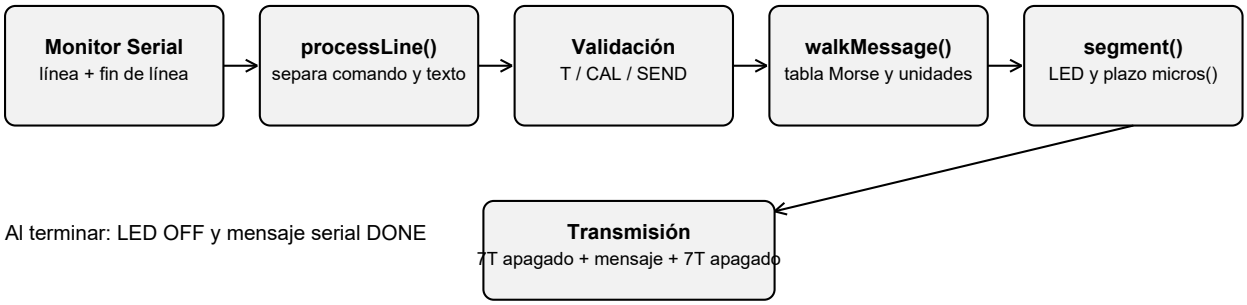
Bloque C — `walkMessage` y `transmit` (líneas 55–89)

`walkMessage` recorre letra por letra; emite 1T para `.`, 3T para `-`, 1T entre símbolos, 3T entre letras y 7T entre palabras. Una primera pasada con `emitir=false` cuenta la duración sin encender el LED. `transmit` anuncia `BEGIN CAL` o `BEGIN SEND` con total de unidades y valor de T, vacía la salida serial, espera un margen OFF de 7T, transmite el texto, espera otro margen OFF de 7T y finalmente anuncia `DONE`. Grabar desde antes de `BEGIN` hasta después de `DONE` conserva la secuencia completa.

Bloque D — `normalizeMessage` y `processLine` (líneas 93–181)

`normalizeMessage` convierte minúsculas a mayúsculas, colapsa espacios repetidos, valida A–Z y rechaza la orden completa si aparece un carácter inválido o supera el límite. `processLine` separa comando y argumentos. `T 300` configura el tiempo base en milisegundos (rango 100–2000); `CAL` transmite exactamente `AN E`; `SEND SIS COM` transmite el texto indicado; `STATUS` solo informa el estado. Una orden desconocida produce `ERROR COMMAND`. El Monitor Serial se configura a 115200 baudios y «Nueva línea».

Figura 8. Ruta de una orden en el Arduino



Bloque E — `setup` y `loop` (líneas 183–214)

`setup` configura salidas, apaga el LED, inicia Serial e imprime `READY`. `loop` acumula caracteres hasta salto de línea, acepta tanto CR como LF sin procesar dos veces CRLF y rechaza líneas desbordadas o con NUL. Tras cada línea válida llama a `processLine`. Por tanto, el Monitor Serial es la **única entrada de órdenes** del transmisor.

4. Programa Python 1: `ejecutar_tarea1.py`

Bloque A — importaciones, rutas y argumentos (líneas 1–102)

El archivo importa funciones de los tres módulos internos y define `PROJECT` como raíz de `tarea1`, base de las salidas predeterminadas. `add_video_arguments` añade `--video`, `--roi X Y ANCHO ALTO`, `--inicio` y `--fin`. Las cuatro funciones `add_*_command` configuran exactamente `patron`, `calibrar`, `decodificar` y `evaluar`; `parser()` registra los comandos y muestra ayuda. `--seleccionar-roi` existe en calibración, aunque el procedimiento fijo del README emplea `--roi` explícita.

Bloque B — `run_pattern` (líneas 103–124)

Normaliza `SIS COM`, llama a `segmentos` para construir la secuencia ideal de ON/OFF con `T=300 ms`, suma unidades y guarda `patron_previsto.csv` y `mensaje.json`. Esta exportación documenta lo **previsto**, no lo que vio la cámara. Para `SIS COM` produce 61T y 18,3 s sin márgenes.

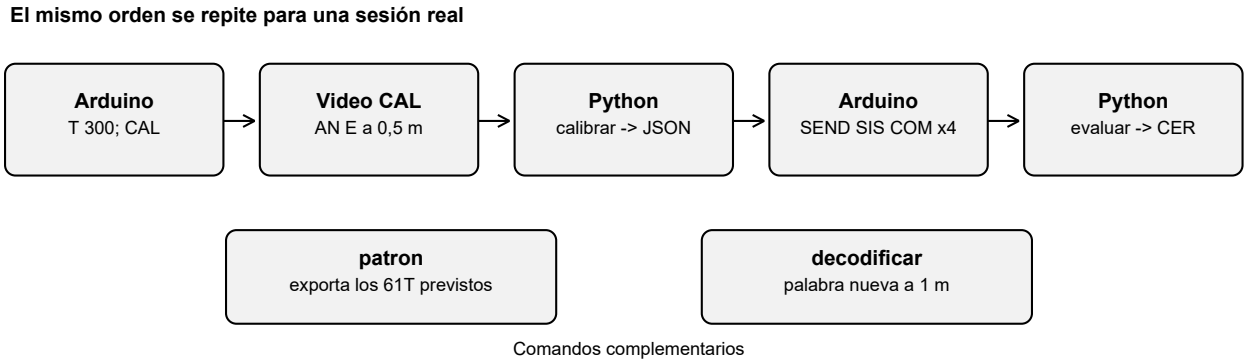
Bloque C — `run_calibration` (líneas 125–147)

Abre el video de `CAL`, mide la intensidad de la ROI en cada cuadro, llama a `calibrate`, incorpora metadatos del video y guarda `resultados/calibracion.json`. Ese JSON contiene `threshold`, `t_seconds`, `tolerance_units`, niveles ON/OFF y ROI. Se obtiene una sola vez a 0,5 m para reutilizarlo sin reajustar umbral según la distancia.

Bloque D — `run_decode`, `run_evaluation` y `main` (líneas 148–207)

`run_decode` carga la calibración y llama a `process_video`; sirve también para la palabra nueva, sin entregarle el texto correcto. `run_evaluation` lee `ensayos.json`, procesa las cuatro distancias e imprime texto y CER de cada una. `main` interpreta los argumentos, fija una carpeta de salida, despacha al bloque correspondiente y devuelve código 2 ante errores de datos o procesamiento. Las rutas de salida explícitas del README evitan carpetas con nombres cambiantes.

Figura 9. Orden único de trabajo para los comandos Python



5. Programa Python 2: `siscom/receptor.py`

Bloque A — `tabla_morse`, `normalize_text`, `encode`, `segmentos` (líneas 1–118)

La tabla directa traduce letra→Morse y la inversa Morse→letra. `normalize_text` acepta únicamente A–Z y espacios; se usa en el **patrón previsto**, no para alterar el texto observado. `encode` muestra la escritura Morse con `/` entre letras y `//` entre palabras. `segmentos` convierte esa escritura en tramos ideales: cada fila tiene estado, unidades, inicio, fin, duración y tipo. Se usa también para conocer la estructura de la calibración `AN E`.

Bloque B — `runs` y `decode` (líneas 119–238)

`runs` compara cada muestra de intensidad con el umbral y agrupa cuadros consecutivos ON/OFF. Cada tramo conserva cantidad de cuadros, tiempo inicial, final y duración. `decode` carga `T_hat` y ε, descarta los márgenes antes del primer ON y después del último ON, divide cada duración por `T_hat` y elige la unidad válida más cercana. Los tramos ON forman `.` o `-`; las pausas OFF terminan letras o insertan espacios. Los casos no válidos producen `?` y advertencias, no una corrección basada en `SIS COM`. Si no detecta pulsos, devuelve texto vacío y una advertencia.

Bloque C — `calibrate`, `save_config`, `load_config` (líneas 239–384)

`calibrate` exige muestra a 0,5 m con valores finitos. Inicializa niveles OFF/ON con percentiles 10 y 90 y reajusta los dos grupos mediante medias hasta estabilizarse; exige contraste de al menos 10 unidades digitales. Tras formar el umbral, recorta márgenes, compara la **secuencia de estados** con la estructura prevista de `AN E` y calcula `T_hat` como mediana de $\Delta t_j / u_{ideal,j}$. La tolerancia ε es el máximo entre 0,35T y el desvío temporal observado más un cuadro expresado en unidades T; se rechaza si llega a 1T. `save_config` serializa el JSON y `load_config` valida versión, distancia, parámetros y ROI antes de procesar otros videos. Esta verificación de secuencia no demuestra por sí sola que todas las duraciones sean correctas; la tolerancia recoge su dispersión.

Bloque D — `extract_video` (líneas 385–fin)

OpenCV abre el archivo y obtiene FPS y número declarado de cuadros. El bucle lee cuadros, selecciona el intervalo pedido, comprueba que la ROI quepa en el cuadro, convierte la ROI a gris y guarda su media como una muestra `x[k]`. También guarda índice de cuadro y tiempo `k/f_s`. Si los timestamps reportados son utilizables, busca variación temporal excesiva; si no, avisa que solo pudo usar FPS nominales. Detecta video inexistente, ausencia de cuadros, lectura incompleta y ROI inválida. En el `finally` libera la captura aun si hubo error.

6. Programa Python 3: `siscom/analisis.py`

Bloque A — figuras de señal y comparación (líneas 1–78)

`signal_figure` dibuja `x[k]` frente a tiempo con línea horizontal en θ; es la evidencia visual de la decisión ON/OFF. `spectrum_figure` muestra FFT y PSD en el intervalo 0 a `f_s/2`, y marca B95. `distance_figure` grafica CER frente a 0,5, 1, 2 y 3 m. `comparison_figure` pone los videos de 0,5 y 3 m en la misma escala vertical, alineados por el primer pulso detectable, con el mismo umbral. Si no hay pulso, `procesar.py` genera una comparación sin esa alineación y lo deja indicado.

Bloque B — `analyze` (líneas 79–157)

Valida muestras y FPS, retira la media, aplica `rfft`, ajusta a amplitud unilateral y calcula su eje de frecuencia. `scipy.signal.welch` estima la PSD unilateral con Hann, hasta 256 muestras por segmento y 50 % de superposición. La regla trapezoidal integra la PSD; `np.interp` encuentra B95 donde la acumulación alcanza 95 %. Guarda además media, varianza, área de PSD y parámetros para hacer reproducible el gráfico.

Bloque C — `cer` (líneas 158–fin)

Construye la matriz de distancia de Levenshtein entre referencia y texto recibido. Cada celda toma el menor costo de sustitución, eliminación o inserción; luego un recorrido inverso cuenta `S`, `D` e `I`. Divide la distancia total por la longitud de la referencia, **incluido el espacio**. El algoritmo puede escoger una de varias alineaciones mínimas y por eso el reparto S/D/I de un caso ambiguo depende de su regla de desempate; el total de errores y el CER permanecen iguales.

7. Programa Python 4: `siscom/procesar.py`

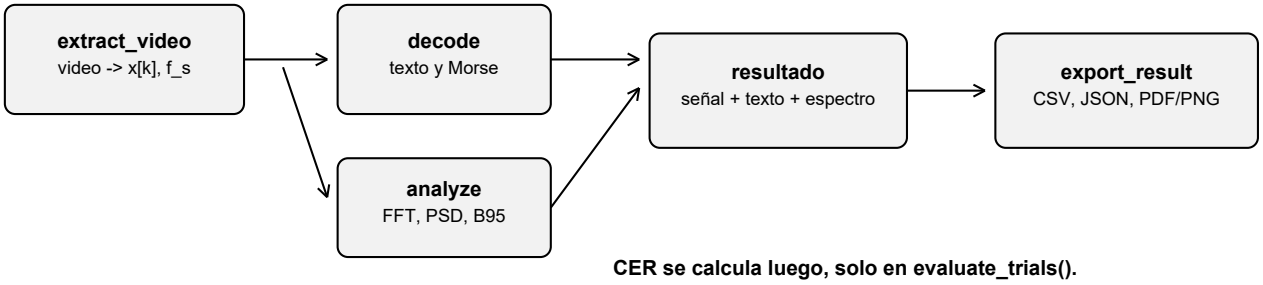
Bloque A — `jsonable`, `write_json`, `write_csv` (líneas 1–55)

Estas funciones exportan datos. `jsonable` convierte arreglos y escalares NumPy a tipos serializables. `write_json` crea carpetas y escribe UTF-8 legible; `write_csv` usa UTF-8 con BOM para apertura sencilla en hojas de cálculo. No toman decisiones de decodificación.

Bloque B — `process_video` (líneas 56–79)

Es la cadena de recepción de un archivo: comprueba que la ROI conserve el tamaño calibrado, llama a `extract_video`, luego a `decode` y a `analyze`. Reúne señal, texto, espectro y configuración; si recibió ruta de salida, llama a `export_result`. `process_video` **no recibe** el texto de referencia.

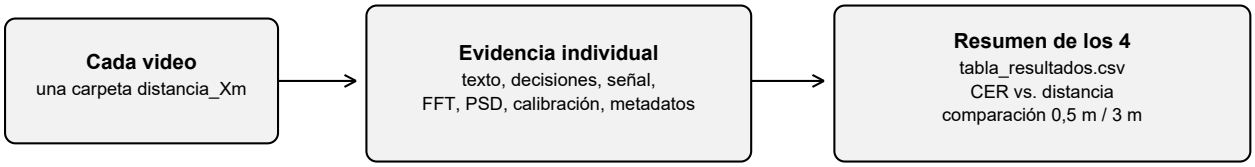
Figura 10. Función process_video y sus entradas y salidas



Bloque C — `export_result` (líneas 80–152)

Guarda `texto_recibido.txt`, decisiones por tramo, curva `senal.csv`, FFT, PSD, parámetros espectrales y gráficos PNG/PDF. Copia la calibración empleada, FPS, tamaño de cuadro, advertencias, versiones de bibliotecas y SHA-256 del video. Esa huella permite comprobar que los resultados corresponden al mismo archivo de entrada. Los gráficos son productos del análisis, no videos originales.

Figura 11. Archivos que exporta el procesamiento



Además se conserva el video original en datos/videos_originales/.

Bloque D — `evaluate_trials` (líneas 153–fin)

Lee `ensayos.json`, exige exactamente las distancias 0,5, 1, 2 y 3 m y resuelve las rutas relativas desde ese archivo. Procesa cada video con la **misma configuración**. Solo después llama a `cer("SIS COM", texto_recibido)`. Registra también ensayos sin adquisición válida o con error, sin inventar un CER. Exporta tabla, gráfico CER-distancia y comparación de señales 0,5/3 m. Si no se detecta pulso para alinear, informa la limitación y guarda un gráfico sin alineación. La bandera `synthetic` identifica pruebas simuladas; los ensayos reales del manifiesto están marcados como `false`.

8. Procedimiento de lectura y ejecución

1. Leer `transmisor_morse.ino` desde `processLine` hacia `transmit` y `walkMessage`. Cargarlo en Arduino UNO; configurar Monitor Serial a 115200 baudios y «Nueva línea». Enviar `T 300`.
2. Para calibrar, grabar desde antes de `CAL` hasta después de `DONE CAL` a 0,5 m. Guardar el video en `datos/calibracion/calibracion.mp4` y elegir una ROI que abarque el LED sin otros focos dominantes.
3. Generar el patrón previsto con `patron`. Calibrar una vez con `calibrar --video ... --roi X Y ANCHO ALTO`; examinar `calibracion.json` y `senal.csv` de los resultados posteriores.
4. Grabar cuatro transmisiones completas de `SEND SIS COM`, una por distancia, manteniendo fuente, cámara, exposición, iluminación y `T`. Guardar los nombres indicados en `ensayos.json` y ejecutar `evaluar --ensayos ensayos.json --config resultados/calibracion.json`.
5. Revisar por distancia `texto_recibido.txt`, `decisiones.csv`, `senal.csv`, `espectro.pdf` y `tabla_resultados.csv`. Verificar que una señal fallida se reporte como fallida; no sustituirla por el texto conocido.
6. Para la palabra nueva, enviarla desde el Monitor Serial, grabar su video a 1 m y usar `decodificar` con el mismo JSON de calibración. La palabra correcta no se entrega como parámetro al receptor.

Los comandos completos y rutas exactas están en `README.md`. Este informe explica lo que hace cada bloque, pero no reemplaza la inspección de las señales reales: enfoque automático, compresión, saturación, otros focos y FPS variable pueden invalidar la medición. Tampoco es correcto llamar a la PSD estimada “respuesta en frecuencia del canal” ni inferir una ley de atenuación óptica solo de cuatro valores CER.

9. Fuentes y alcance

El enunciado `Tarea1_SisCom_2026.docx` fija el procedimiento; `Modelos análogos de Canales de Comunicaciones` aporta `s(t), n(t), r(t)`; `BW` y `Nyquist`, muestreo; `Series y transformadas de Fourier`, la FFT; `Respuesta espectral SLTI`, la PSD y Parseval. Los originales están en `teoria/originales/`. El modelo `r_d(t)` adapta la notación para explicar el montaje y no es una medición del programa.

Anexo: código explicado con listados tipo lstlisting

Los recuadros siguientes usan formato tipo lstlisting: letra monoespaciada, marco y numeración original de líneas. Cada recuadro se genera directamente desde el archivo fuente que indica su título. La explicación inmediatamente posterior sigue el mismo orden que el código.

El flujo completo es Monitor Serial -> Arduino/LED -> video -> x[k] -> umbral y duraciones -> texto reconstruido -> análisis espectral -> comparación CER. La referencia SIS COM se usa únicamente al final, para evaluar el resultado.

Arduino UNO: transmisor_morse.ino

Memoria, parámetros y tabla Morse

lstlisting - Memoria fija y tabla Morse (transmisor_morse.ino, lineas 1-26; C++)

```
#include <Arduino.h>
#include <string.h>

// ------------------------------------------------------------------------
// Configuración y memoria fija
// ------------------------------------------------------------------------
// UNO clasico (ATmega328P). Sin String ni asignacion dinamica.
const uint8_t PIN_LED = 8;
const bool SINCRONIZAR_LED_INTEGRADO = true;  // false para desactivar el indicador integrado.
const uint8_t MAXIMO_LETRAS = 80;
const uint16_t CAPACIDAD_LINEA = 192;  // Incluye terminador NUL.
char bufferLinea[CAPACIDAD_LINEA];
char bufferMensaje[MAXIMO_LETRAS + 1];  // 80 caracteres normalizados + NUL.
uint16_t longitudLinea = 0;
bool lineaDesbordada = false;
bool retornoCarroPrevio = false;
uint16_t duracionUnidadMs = 300;

// ------------------------------------------------------------------------
// Tabla Morse: el índice 0 representa A y el índice 25 representa Z
// ------------------------------------------------------------------------
const char CODIGO_MORSE[26][5] = {
  ".-", "-...", "-.-.", "-..", ".", "..-.", "--.", "....", "..",
  ".---", "-.-", ".-..", "--", "-.", "---", ".--.", "--.-", ".-.",
  "...", "-", "..-", "...-", ".--", "-..-", "-.--", "--.."
};
```

1. Se incluyen las funciones de Arduino y utilidades de cadenas de C. Los buffers fijos evitan asignación dinámica en el UNO.
2. PIN_LED fija la salida externa en D8; duracionUnidadMs establece inicialmente T = 300 ms. bufferLinea recibe el comando serial y bufferMensaje guarda el texto validado.
3. CODIGO_MORSE contiene las 26 letras en orden alfabético. Para una letra, su índice es letra - 'A'; los espacios se procesan como pausas y no como letras.

Conmutar el LED y recorrer símbolos

lstlisting - Funciones setLed, segment y walkMessage (transmisor_morse.ino, lineas 31-69; C++)

```
// Reconoce espacios y tabulaciones admitidos como separadores en las órdenes.
bool esEspacio(char caracter) {
  return caracter == ' ' || caracter == '\t' || caracter == '\v' || caracter == '\f';
}

void setLed(bool encendido) {
  digitalWrite(PIN_LED, encendido ? HIGH : LOW);
  if (SINCRONIZAR_LED_INTEGRADO) digitalWrite(LED_BUILTIN, encendido ? HIGH : LOW);
}

// Cuenta o emite los segmentos Morse. Las pausas son 1T dentro de una letra,
// 3T entre letras y 7T entre palabras.
// Los plazos acumulados evitan sumar el coste de digitalWrite a cada segmento.
```

lstlisting - Funciones setLed, segment y walkMessage (transmisor_morse.ino, lineas 31-69; C++)

```
44 void segment(bool encendido, uint8_t unidades, bool emitir,
45              uint16_t &totalUnidades, unsigned long &plazo) {
46   totalUnidades += unidades;
47   if (!emitir) return;
48   setLed(encendido);
49   plazo += static_cast<unsigned long>(unidades) * duracionUnidadMs * 1000UL;
50   while (static_cast<long>(micros() - plazo) < 0) {
51     // Interrupciones habilitadas; ningun print durante la secuencia.
52   }
53 }
54
55 uint16_t walkMessage(const char *texto, bool emitir, unsigned long &plazo) {
56   uint16_t totalUnidades = 0;
57   for (uint16_t indiceLetra = 0; texto[indiceLetra]; ++indiceLetra) {
58     if (texto[indiceLetra] == ' ') continue;
59     const char *codigo = CODIGO_MORSE[texto[indiceLetra] - 'A'];
60     for (uint8_t indiceElemento = 0; codigo[indiceElemento]; ++indiceElemento) {
61       segment(true, codigo[indiceElemento] == '.' ? 1 : 3, emitir, totalUnidades, plazo);
62       if (codigo[indiceElemento + 1]) segment(false, 1, emitir, totalUnidades, plazo);
63     }
64     if (texto[indiceLetra + 1]) {
65       segment(false, texto[indiceLetra + 1] == ' ' ? 7 : 3, emitir, totalUnidades, plazo);
66     }
67   }
68   return totalUnidades;
69 }
```

1. esEspacio reconoce separadores. setLed cambia el pin externo y, si está habilitado, el LED integrado.
2. segment acumula las unidades. En modo de conteo no enciende el LED; en transmisión lo conmuta y espera hasta un plazo medido con micros().
3. Los plazos se acumulan segmento a segmento; así, el tiempo de digitalWrite no se agrega como retraso extra en cada tramo.
4. walkMessage consulta la tabla y emite 1T para punto, 3T para raya, 1T entre símbolos, 3T entre letras y 7T entre palabras.

Añadir márgenes al mensaje

lstlisting - Función transmit (transmisor_morse.ino, lineas 71-86; C++)

```
71 void transmit(const char *texto, bool esCalibracion) {
72   unsigned long plazo = 0;
73   const uint16_t totalUnidades = walkMessage(texto, false, plazo);
74   Serial.print(esCalibracion ? F("BEGIN CAL ") : F("BEGIN SEND "));
75   Serial.print(totalUnidades);
76   Serial.print(' ');
77   Serial.println(duracionUnidadMs);
78   Serial.flush();  // Vacía el mensaje serial antes de empezar a medir el tiempo.
79   plazo = micros();
80   uint16_t unidadesMargen = 0;
81   segment(false, 7, true, unidadesMargen, plazo);
82   walkMessage(texto, true, plazo);
83   segment(false, 7, true, unidadesMargen, plazo);
84   setLed(false);
85   Serial.println(esCalibracion ? F("DONE CAL") : F("DONE SEND"));
86 }
```

1. El primer recorrido cuenta cuántas unidades ocupa el mensaje sin emitir luz. El Arduino anuncia BEGIN CAL o BEGIN SEND, las unidades y el valor de T.
2. Serial.flush() espera que el anuncio salga antes de iniciar los tiempos luminosos.
3. Se inicia el reloj, se transmite un margen apagado de 7T, el patrón y otro margen de 7T. Al final se apaga el LED y se imprime DONE.
4. El video debe cubrir el intervalo completo, incluidos ambos márgenes.

Normalizar y validar el mensaje

lstlisting - Función normalizeMessage (transmisor_morse.ino, lineas 91-126; C++)

```
91  // Convierte minúsculas y separadores repetidos a la forma aceptada.
92  // Valida todo el mensaje antes de transmitir; los signos no se eliminan.
93  bool normalizeMessage(const char *entrada) {
94    uint8_t cantidadLetras = 0;
95    uint16_t indiceSalida = 0;
96    bool espacioPendiente = false;
97    for (; *entrada; ++entrada) {
98      char caracter = *entrada;
99      if (esEspacio(caracter)) {
100       if (indiceSalida) espacioPendiente = true;
101       continue;
102     }
103     if (caracter >= 'a' && caracter <= 'z') caracter -= 'a' - 'A';
104     if (caracter < 'A' || caracter > 'Z') {
105       Serial.println(F("ERROR INVALID_CHARACTER"));
106       return false;
107     }
108     if (++cantidadLetras > MAXIMO_LETRAS) {
109       Serial.println(F("ERROR TEXT_TOO_LONG"));
110       return false;
111     }
112     if (indiceSalida + (espacioPendiente ? 1 : 0) + 1 > MAXIMO_LETRAS) {
113       Serial.println(F("ERROR TEXT_TOO_LONG"));
114       return false;
115     }
116     if (espacioPendiente) bufferMensaje[indiceSalida++] = ' ';
117     bufferMensaje[indiceSalida++] = caracter;
118     espacioPendiente = false;
119   }
120   bufferMensaje[indiceSalida] = '\0';
121   if (!cantidadLetras) {
122     Serial.println(F("ERROR EMPTY_TEXT"));
123     return false;
124   }
125   return true;
126 }
```

1. La función recorre la entrada y deja pendiente un solo espacio cuando encuentra varios separadores seguidos.
2. Convierte minúsculas a mayúsculas. Si aparece algo que no sea A-Z, informa el error y detiene la orden; no borra números ni signos para aceptarlos.
3. Rechaza más de 80 letras o un texto que no cabe en el buffer.
4. Termina el texto con el carácter nulo. Si no queda ninguna letra, informa ERROR EMPTY_TEXT.

Interpretar SEND, T, CAL y STATUS

lstlisting - Función processLine (transmisor_morse.ino, lineas 128-178; C++)

```
128 // ------------------------------------------------------------------------
129 // Parser serial: separa el comando de sus argumentos y los valida
130 // ------------------------------------------------------------------------
131 void processLine() {
132   bufferLinea[longitudLinea] = '\0';
133   char *comando = bufferLinea;
134   while (esEspacio(*comando)) ++comando;
135   char *finComando = comando + strlen(comando);
136   while (finComando > comando && esEspacio(finComando[-1])) --finComando;
137   *finComando = '\0';
138   if (!*comando) return;  // Lineas vacias ignoradas.
139   char *argumentos = comando;
140   while (*argumentos && !esEspacio(*argumentos)) ++argumentos;
141   if (*argumentos) *argumentos++ = '\0';
142   while (esEspacio(*argumentos)) ++argumentos;
143
144   if (strcmp(comando, "SEND") == 0) {
145     if (normalizeMessage(argumentos)) transmit(bufferMensaje, false);
146   } else if (strcmp(comando, "T") == 0) {
147     uint16_t valor = 0;
148     bool valorValido = *argumentos != '\0';
149     for (const char *digito = argumentos; *digito; ++digito) {
150       if (*digito < '0' || *digito > '9' || valor > 200) {
```

lstlisting - Función processLine (transmisor_morse.ino, lineas 128-178; C++)

```
        valorValido = false;
        break;
      }
      valor = valor * 10 + (*digito - '0');
    }
    if (!valorValido || valor < 100 || valor > 2000) {
      Serial.println(F("ERROR T_RANGE"));
    } else {
      duracionUnidadMs = valor;
      Serial.print(F("OK T "));
      Serial.println(duracionUnidadMs);
    }
  } else if (strcmp(comando, "CAL") == 0 || strcmp(comando, "STATUS") == 0) {
    if (*argumentos) {
      Serial.println(F("ERROR ARGUMENTS"));
    } else if (strcmp(comando, "CAL") == 0) {
      transmit("AN E", true);
    } else {
      Serial.print(F("STATUS T "));
      Serial.print(duracionUnidadMs);
      Serial.print(F(" PIN 8 BUILTIN "));
      Serial.print(SINCRONIZAR_LED_INTEGRADO ? 1 : 0);
      Serial.println(F(" MAX_LETTERS 80 IDLE"));
    }
  } else {
    Serial.println(F("ERROR COMMAND"));
  }
}
```

1. Primero recorta espacios de los extremos; luego separa el primer grupo como comando y el resto como argumentos.
2. SEND valida el texto antes de transmitir. T admite dígitos y limita la unidad a 100-2000 ms.
3. CAL transmite la secuencia conocida AN E. STATUS informa T y la configuración del LED.
4. Comandos desconocidos, argumentos incorrectos o tiempos fuera de rango producen ERROR sin emitir el patrón.

Inicializar y recibir líneas por Serial

lstlisting - Funciones setup y loop (transmisor_morse.ino, lineas 180-215; C++)

```
// ------------------------------------------------------------------------------
// Inicialización y recepción continua
// ------------------------------------------------------------------------------
void setup() {
  pinMode(PIN_LED, OUTPUT);
  if (SINCRONIZAR_LED_INTEGRADO) pinMode(LED_BUILTIN, OUTPUT);
  setLed(false);
  Serial.begin(115200);
  Serial.println(F("READY"));
}

void loop() {
  while (Serial.available() > 0) {
    const char c = static_cast<char>(Serial.read());
    if (c == '\n' && retornoCarroPrevio) {
      retornoCarroPrevio = false;
      continue;
    }
    retornoCarroPrevio = c == '\r';
    if (c == '\r' || c == '\n') {
      if (lineaDesbordada) Serial.println(F("ERROR LINE_OVERFLOW"));
      else processLine();
      longitudLinea = 0;
      lineaDesbordada = false;
    } else if (!lineaDesbordada) {
      if (c == '\0') {
        // NUL no puede truncar silenciosamente una orden valida.
        lineaDesbordada = true;
      } else if (longitudLinea < CAPACIDAD_LINEA - 1) {
        bufferLinea[longitudLinea++] = c;
      } else {
```

lstlisting - Funciones setup y loop (transmisor_morse.ino, lineas 180-215; C++)

```
        lineaDesbordada = true;  // Descarta toda la línea hasta CR/LF.
      }
    }
  }
}
```

1. setup configura salidas, apaga los LED, inicia el Monitor Serial a 115200 baudios e imprime READY.
2. loop acumula caracteres hasta CR o LF. retornoCarroPrevio evita procesar dos veces un fin de línea CRLF.
3. El buffer tiene tamaño máximo. Si encuentra NUL o desbordamiento, descarta la línea hasta el próximo fin de línea.
4. Una línea completa llega a processLine; por eso las órdenes se escriben directamente en el Monitor Serial del IDE.

Interfaz de comandos: ejecutar_tarea1.py

Importaciones, raíz y argumentos de video

lstlisting - Dependencias y opciones comunes (ejecutar_tarea1.py, lineas 14-38; Python)

```
import argparse
from datetime import datetime
from pathlib import Path
import sys

from siscom.receptor import (
    calibrate, encode, extract_video, load_config, normalize_text,
    save_config, segmentos,
)
from siscom.procesar import evaluate_trials, process_video, write_csv, write_json


# La carpeta de la tarea es la base para las rutas por defecto.
PROJECT = Path(__file__).resolve().parents[2]


def add_video_arguments(command_parser):
    """Añade las opciones compartidas por calibrar y decodificar."""
    command_parser.add_argument("--video", required=True, type=Path)
    command_parser.add_argument(
        "--roi", nargs=4, type=int, metavar=("X", "Y", "ANCHO", "ALTO")
    )
    command_parser.add_argument("--inicio", type=float, default=0.0)
    command_parser.add_argument("--fin", type=float)

```

1. argparse interpreta comandos; Path resuelve rutas; datetime genera carpetas únicas y sys entrega errores al terminal.
2. Las funciones importadas desde siscom conectan esta interfaz con el receptor y procesador; aquí no se duplica el algoritmo.
3. PROJECT resuelve la carpeta raíz de tarea1, por lo que las salidas predeterminadas no dependen de dónde se abrió PowerShell.
4. add_video_arguments agrega video, ROI y límites temporales en segundos. La ROI tiene x, y, ancho y alto.

Registrar patrón y calibración

lstlisting - Comandos patron y calibrar (ejecutar_tarea1.py, lineas 40-61; Python)

```
def add_pattern_command(commands):
    """Registra el comando que exporta el patrón Morse ideal."""
    command_parser = commands.add_parser(
        "patron", help="Exportar patron Morse previsto"
    )
    command_parser.add_argument("--texto", default="SIS COM")
    command_parser.add_argument("--t-ms", type=int, default=300)
    command_parser.add_argument("--salida", type=Path)


```

lstlisting - Comandos patron y calibrar (ejecutar_tarea1.py, lineas 40-61; Python)

```python
def add_calibration_command(commands):
    """Registra la calibración inicial a 0,5 m."""
    command_parser = commands.add_parser(
        "calibrar", help="Calibrar una vez a 0,5m con video CAL AN E"
    )
    add_video_arguments(command_parser)
    command_parser.add_argument("--seleccionar-roi", action="store_true")
    command_parser.add_argument(
        "--salida", type=Path,
        default=PROJECT / "resultados/calibracion.json",
    )

```

1. add_pattern_command define patron; el texto por defecto es SIS COM y T por defecto es 300 ms.
2. add_calibration_command define calibrar y reutiliza las opciones de video.
3. Se puede dar una ROI con coordenadas o pedir su selección interactiva.
4. Sin otra ruta, la calibración se guarda en resultados/calibracion.json.

Registrar decodificar, evaluar y ayuda

lstlisting - Comandos decodificar y evaluar (ejecutar_tarea1.py, lineas 63-100; Python)

```python
def add_decode_command(commands):
    """Registra la recepción y decodificación de un video."""
    command_parser = commands.add_parser(
        "decodificar", help="Reconstruir texto solo a partir del video"
    )
    add_video_arguments(command_parser)
    command_parser.add_argument(
        "--config", type=Path,
        default=PROJECT / "resultados/calibracion.json",
    )
    command_parser.add_argument("--salida", type=Path)


def add_evaluation_command(commands):
    """Registra el procesamiento de ensayos a distintas distancias."""
    command_parser = commands.add_parser(
        "evaluar", help="Procesar cuatro distancias y calcular CER despues"
    )
    command_parser.add_argument("--ensayos", type=Path, required=True)
    command_parser.add_argument(
        "--config", type=Path,
        default=PROJECT / "resultados/calibracion.json",
    )
    command_parser.add_argument("--salida", type=Path)


def parser():
    """Construye el parser principal y registra todos los subcomandos."""
    root_parser = argparse.ArgumentParser(
        description="Enlace optico Morse Arduino + Python"
    )
    commands = root_parser.add_subparsers(dest="command", required=True)

    add_pattern_command(commands)
    add_calibration_command(commands)
    add_decode_command(commands)
    add_evaluation_command(commands)
    return root_parser
```

1. decodificar recibe un video y carga la configuración de calibración.
2. evaluar lee el manifiesto con los cuatro ensayos y usa la misma configuración.
3. parser registra los cuatro subcomandos y exige que el usuario elija uno.
4. --help presenta opciones distintas para cada tarea.

Generar el patrón previsto

lstlisting - Función run_pattern (ejecutar_tarea1.py, lineas 103-123; Python)

```python
def run_pattern(args, output_path):
    """Guarda el patrón temporal ideal para el texto solicitado."""
    text = normalize_text(args.texto)
    pattern_rows = segmentos(text, args.t_ms / 1000)
    total_units = sum(row["units"] for row in pattern_rows)
    duration_seconds = pattern_rows[-1]["end"]

    write_csv(output_path / "patron_previsto.csv", pattern_rows)
    write_json(
        output_path / "mensaje.json",
        dict(
            text=text,
            morse=encode(text),
            t_ms=args.t_ms,
            units=total_units,
            duration=duration_seconds,
        ),
    )
    print(encode(text))
    print(f"{total_units}T; {duration_seconds:.3f} s")

```

1. normalize_text valida el texto; segmentos lo convierte en tramos ideales. args.t_ms / 1000 pasa T de milisegundos a segundos.
2. Se suman las unidades T y se toma el final del último tramo como duración.
3. El CSV guarda tramos previstos y el JSON guarda texto, Morse, T y duración.
4. Estos archivos representan el patrón ideal, no mediciones de cámara.

Calibrar el video CAL

lstlisting - Función run_calibration (ejecutar_tarea1.py, lineas 125-146; Python)

```python
def run_calibration(args, output_path):
    """Extrae la señal CAL del video y guarda su configuración."""
    video_data = extract_video(
        args.video,
        roi=args.roi,
        inicio=args.inicio,
        fin=args.fin,
        seleccion_interactiva=args.seleccionar_roi,
    )
    config = calibrate(
        video_data["intensity"],
        video_data["fs"],
        video_data["roi"],
        fuente=args.video,
    )
    config["video_metadata"] = video_data["metadata"]
    save_config(config, output_path)
    print(
        f"Umbral {config['threshold']:.2f}; T {config['t_seconds']:.4f} s; "
        f"tolerancia {config['tolerance_units']:.3f}T"
    )

```

1. extract_video entrega intensidad por cuadro, FPS, ROI y metadatos.
2. calibrate usa esa señal para estimar umbral, T y tolerancia. La calibración oficial se fija a 0,5 m.
3. Los metadatos se agregan al resultado y save_config escribe el JSON.
4. En pantalla se resumen el umbral, T estimada y tolerancia.

Decodificar o evaluar

lstlisting - Funciones run_decode y run_evaluation (ejecutar_tarea1.py, lineas 148-177; Python)

```python
def run_decode(args, output_path):
```

lstlisting - Funciones run_decode y run_evaluation (ejecutar_tarea1.py, lineas 148-177; Python)

```python
    """Decodifica el video usando los parámetros de calibración guardados."""
    result_data = process_video(
        args.video,
        load_config(args.config),
        roi=args.roi,
        inicio=args.inicio,
        fin=args.fin,
        salida=output_path,
    )
    result = result_data["result"]
    print(f"Texto recibido: {result['text']!r}")
    print(f"Morse: {result['morse']}")
    for warning in result["warnings"]:
        print("Aviso:", warning)


def run_evaluation(args, output_path):
    """Evalúa los ensayos y devuelve 2 si hubo errores de procesamiento."""
    rows = evaluate_trials(args.ensayos, load_config(args.config), output_path)
    for row in rows:
        print(
            f"{row['distance_m']:g} m: {row['text']!r}, "
            f"CER={row['cer']}, {row['status']}"
        )

    if any(row["status"] == "error de procesamiento" for row in rows):
        print(f"Resultados parciales en {output_path}", file=sys.stderr)
        return 2
    return None
```

1. run_decode carga el JSON y llama a process_video; muestra texto, Morse y avisos. No pasa la palabra correcta al receptor.
2. run_evaluation llama a evaluate_trials, que procesa los cuatro videos y retorna sus filas.
3. Si hubo un error de procesamiento, se informa dónde quedaron resultados parciales y se devuelve código 2.

Despachar el subcomando y responder errores

lstlisting - Función main (ejecutar_tarea1.py, lineas 180-207; Python)

```python
def main(argv=None):
    """Interpreta los argumentos, ejecuta el subcomando y devuelve su estado."""
    args = parser().parse_args(argv)
    default_output = PROJECT / "resultados" / (
        args.command + "_" + datetime.now().strftime("%Y%m%d_%H%M%S_%f")
    )
    output_path = getattr(args, "salida", None) or default_output

    try:
        if args.command == "patron":
            run_pattern(args, output_path)
        elif args.command == "calibrar":
            run_calibration(args, output_path)
        elif args.command == "decodificar":
            run_decode(args, output_path)
        elif args.command == "evaluar":
            status = run_evaluation(args, output_path)
            if status is not None:
                return status
        print(f"Salida: {output_path.resolve()}")
        return 0
    except (ValueError, OSError, KeyError) as exc:
        print(f"Error: {exc}", file=sys.stderr)
        return 2


if __name__ == "__main__":
    sys.exit(main())
```

1. parser().parse_args transforma los argumentos de consola en opciones del comando elegido.
2. Si no se definió salida, main crea una carpeta con el comando y fecha/hora.

3. Una cadena if/elif llama a la función correspondiente: patrón, calibración, decodificación o evaluación.
4. Los errores de datos o archivos se imprimen en stderr y el proceso termina con estado 2; una ejecución normal retorna 0.

Medición, calibración y decodificación: siscom/receptor.py

Tabla Morse y validación del texto

lstlisting - Tabla Morse y normalize_text (receptor.py, lineas 1-55; Python)

```python
"""Lectura del video, calibración y decodificación Morse."""

# 1. Código Morse y patrón temporal esperado

from math import isfinite

# Cada patrón corresponde a la letra que ocupa la misma posición del alfabeto.
# El espacio no está en esta tabla: separa palabras y se trata por separado.
alfabeto = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
tabla_morse = dict(zip(
    alfabeto,
    [
        ".-", "-...", "-.-.", "-..", ".", "..-.", "--.", "....", "..", ".---",
        "-.-", ".-..", "--", "-.", "---", ".--.", "--.-", ".-.", "...", "-",
        "..-", "...-", ".--", "-..-", "-.--", "--..",
    ],
))

# Tabla inversa: permite traducir un patrón Morse a su letra correspondiente.
tabla_morse_inversa = {patron: letra for letra, patron in tabla_morse.items()}

# Mensaje conocido que el transmisor puede enviar para la calibración.
texto_calibracion = "AN E"


def normalize_text(texto: str) -> str:
    """Valida y normaliza texto para transmitirlo en Morse.

    Solo admite letras ASCII A-Z y espacios. Convierte las letras a mayúsculas,
    reduce espacios repetidos y en los extremos, y limita el mensaje a 80
    caracteres normalizados. Esta función es para la entrada del transmisor;
    no se aplica al texto que el receptor reconstruye.
    """
    caracteres_permitidos = "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz "
    if any(caracter not in caracteres_permitidos for caracter in texto):
        raise ValueError("Solo se admiten letras A-Z y espacios; sin tildes ni numeros")

    texto_mayusculas = texto.upper()
    if any(
        caracter not in tabla_morse and caracter != " "
        for caracter in texto_mayusculas
    ):
        raise ValueError("Solo se admiten letras A-Z y espacios; sin tildes ni numeros")

    # Se conserva la normalización original: la segunda operación descarta
    # los segmentos vacíos y vuelve a unir las palabras con un solo espacio.
    texto_normalizado = " ".join(texto_mayusculas.split(" "))
    texto_normalizado = " ".join(
        palabra for palabra in texto_normalizado.split(" ") if palabra
    )
    if not texto_normalizado:
        raise ValueError("El mensaje no puede estar vacio")
    if len(texto_normalizado) > 80:
        raise ValueError("Maximo 80 caracteres normalizados")
    return texto_normalizado
```

1. tabla_morse traduce letras a punto-raya; tabla_morse_inversa hace la traducción contraria.
2. texto_calibracion fija AN E como secuencia conocida.
3. normalize_text acepta letras A-Z y espacios, pasa a mayúsculas, reduce espacios repetidos y valida longitud.
4. Se normaliza el patrón de entrada, no el texto que el receptor reconstruye.

Convertir texto a Morse legible

lstlisting - Función encode (receptor.py, lineas 58-72; Python)

```python
def encode(texto: str) -> str:
    """Traduce texto válido a Morse con separadores legibles.

    Usa `/` entre letras y `//` entre palabras. La validación y normalización
    se realizan primero mediante :func:`normalize_text`.
    """
    texto_normalizado = normalize_text(texto)
    palabras = texto_normalizado.split(" ")

    palabras_en_morse = []
    for palabra in palabras:
        letras_en_morse = [tabla_morse[letra] for letra in palabra]
        palabras_en_morse.append(" / ".join(letras_en_morse))

    return " // ".join(palabras_en_morse)
```

1. Se valida el texto antes de consultar la tabla.
2. Se separan palabras por espacios y se buscan los códigos Morse de cada letra.
3. / separa letras y // palabras para imprimir; no son señales ópticas.

Construir tramos Morse ideales

lstlisting - Función segmentos (receptor.py, lineas 75-117; Python)

```python
def segmentos(texto: str, duracion_t: float = 0.3) -> list[dict]:
    """Devuelve los tramos ON/OFF ideales de un mensaje.

    El tiempo se expresa en segundos y cada duración conserva también su
    equivalente en unidades T. No incluye los márgenes apagados del video.
    """
    if not isfinite(duracion_t) or duracion_t <= 0:
        raise ValueError("T debe ser positivo y finito")

    texto = normalize_text(texto)
    tramos = []
    tiempo_actual = 0.0

    def agregar(estado: int, unidades: int, tipo: str):
        nonlocal tiempo_actual
        tiempo_final = tiempo_actual + unidades * duracion_t
        tramos.append(dict(
            state=estado,
            units=unidades,
            start=tiempo_actual,
            end=tiempo_final,
            duration=tiempo_final - tiempo_actual,
            kind=tipo,
        ))
        tiempo_actual = tiempo_final

    for indice_letra, letra in enumerate(texto):
        if letra == " ":
            continue

        codigo = tabla_morse[letra]
        for indice_simbolo, simbolo in enumerate(codigo):
            es_punto = simbolo == "."
            agregar(1, 1 if es_punto else 3, "punto" if es_punto else "raya")
            if indice_simbolo < len(codigo) - 1:
                agregar(0, 1, "simbolo")

        if indice_letra < len(texto) - 1:
            separa_palabras = texto[indice_letra + 1] == " "
            agregar(0, 7 if separa_palabras else 3,
                    "palabra" if separa_palabras else "letra")

    return tramos
```

1. Se exige T finito y positivo y se inicia el reloj lógico en cero.

2. agregar guarda estado ON/OFF, unidades, inicio, fin, duración y tipo.

3. Punto = ON de 1T; raya = ON de 3T; separación interna = OFF de 1T.

4. Entre letras se agregan 3T OFF y entre palabras 7T OFF. No se incluyen márgenes de video.

Umbralizar y agrupar cuadros

lstlisting - Función runs (receptor.py, lineas 119-154; Python)

```python
# 2. Umbral, tramos ON/OFF y decodificación

import numpy as np


def runs(intensidad, fps: float, umbral: float) -> list[dict]:
    """Agrupa cuadros consecutivos según el LED esté apagado o encendido.

    Cada resultado conserva los índices temporales como segundos, para que
    las duraciones se puedan comparar con la unidad Morse T.
    """
    intensidades = np.asarray(intensidad, dtype=float)
    if intensidades.ndim != 1 or not np.all(np.isfinite(intensidades)):
        raise ValueError("La intensidad debe ser un vector finito")
    if not np.isfinite(fps) or fps <= 0 or not np.isfinite(umbral):
        raise ValueError("FPS/umbral invalidos")
    if not len(intensidades):
        return []
    # Un valor sobre el umbral representa luz encendida. Los cambios de
    # estado marcan los límites de cada tramo continuo.
    led_encendido = intensidades > umbral
    limites_tramos = np.r_[
        0,
        np.flatnonzero(led_encendido[1:] != led_encendido[:-1]) + 1,
        len(led_encendido),
    ]
    return [
        dict(
            state=int(led_encendido[cuadro_inicial]),
            start=cuadro_inicial / fps,
            end=cuadro_final / fps,
            duration=(cuadro_final - cuadro_inicial) / fps,
            frames=int(cuadro_final - cuadro_inicial),
        )
        for cuadro_inicial, cuadro_final in zip(limites_tramos[:-1], limites_tramos[1:])
    ]
```

1. La señal se convierte a vector numérico y se comprueba que sea finita; FPS y umbral también deben ser válidos.

2. intensidades > umbral produce la señal binaria: verdadero es ON y falso es OFF.

3. Los índices donde cambia el estado delimitan cada racha continua.

4. Índice dividido por FPS da tiempo aproximado; cada racha conserva duración y cantidad de cuadros.

Preparar decode y cerrar letras

lstlisting - Inicio de decode (receptor.py, lineas 157-186; Python)

```python
def decode(intensidad, fps: float, configuracion: dict) -> dict:
    """Decodifica las rachas de luz usando la calibración indicada.

    Los márgenes sin luz se descartan; luego se clasifican las duraciones
    como puntos, rayas o separaciones y se traducen los patrones a letras.
    """
    unidad_t = float(configuracion["t_seconds"])
    tolerancia = float(configuracion["tolerance_units"])
    if not np.isfinite(unidad_t) or unidad_t <= 0 or not 0 < tolerancia < 1:
        raise ValueError("T/tolerancia invalidos: tolerancia en unidades T debe ser <1")
    todos_los_tramos = runs(intensidad, fps, float(configuracion["threshold"]))
    indices_activos = [
        indice for indice, tramo in enumerate(todos_los_tramos) if tramo["state"]
    ]
    if not indices_activos:
        return dict(text="", morse="", decisions=[], duration=0., warnings=["Sin pulsos detectados"])
```

lstlisting - Inicio de decode (receptor.py, lineas 157-186; Python)

```python
    # Ignora el silencio anterior al primer pulso y posterior al último.
    tramos_seleccionados = todos_los_tramos[indices_activos[0] : indices_activos[-1] + 1]
    caracteres_decodificados, patrones_decodificados, decisiones = [], [], []
    patron_actual = ""
    advertencias = []

    def finish_letter():
        """Convierte el patrón acumulado y prepara el siguiente carácter."""
        nonlocal patron_actual
        if patron_actual:
            caracteres_decodificados.append(tabla_morse_inversa.get(patron_actual, "?"))
            patrones_decodificados.append(patron_actual)
            patron_actual = ""

```

1. Se leen del JSON T estimada, tolerancia y umbral; valores fuera de rango se rechazan.
2. runs convierte las muestras en rachas ON/OFF.
3. Sin pulsos, se retorna texto vacío y un aviso. Si hay pulsos, se recortan silencios externos.
4. finish_letter consulta la tabla inversa; un patrón inexistente se representa con ?.

Clasificar duraciones y formar el texto

lstlisting - Decisiones temporales en decode (receptor.py, lineas 187-237; Python)

```python
    for tramo in tramos_seleccionados:
        # Se expresa cada racha como múltiplo de T y se elige la duración
        # estándar más cercana, siempre que caiga dentro de la tolerancia.
        unidades_medidas = tramo["duration"] / unidad_t
        unidades_permitidas = [1, 3] if tramo["state"] else [1, 3, 7]
        unidad_mas_cercana = min(
            unidades_permitidas,
            key=lambda unidad: abs(unidades_medidas - unidad),
        )
        valido = abs(unidades_medidas - unidad_mas_cercana) <= tolerancia + 1e-9
        if tramo["state"]:
            tipo = ("punto" if unidad_mas_cercana == 1 else "raya") if valido else "pulso_invalido"
            patron_actual += ("." if unidad_mas_cercana == 1 else "-") if valido else "!"
        elif not valido:
            tipo = "pausa_invalida"
            finish_letter()
            caracteres_decodificados.append("?")
            patrones_decodificados.append("[pausa?]")
        elif unidad_mas_cercana == 3:
            tipo = "letra"
            finish_letter()
        elif unidad_mas_cercana == 7:
            tipo = "palabra"
            finish_letter()
            caracteres_decodificados.append(" ")
            patrones_decodificados.append("/")
        else:
            tipo = "simbolo"
        if not valido:
            advertencias.append(
                f"{tipo} a {tramo['start']:.3f} s ({unidades_medidas:.2f}T)"
            )
        decisiones.append(
            {
                **tramo,
                "units_measured": unidades_medidas,
                "kind": tipo,
                "valid": valido,
                "pattern": patron_actual,
            }
        )

    finish_letter()
    morse = " / ".join(patrones_decodificados).replace(" / / / ", " // ")
    return dict(
        text="".join(caracteres_decodificados),
```

lstlisting - Decisiones temporales en decode (receptor.py, lineas 187-237; Python)

```
        morse=morse,
        decisions=decisiones,
        duration=tramos_seleccionados[-1]["end"] - tramos_seleccionados[0]["start"],
        warnings=advertencias,
    )
```

1. Para cada racha se calcula unidades medidas = duración / T.
2. Para ON se permiten 1T y 3T; para OFF, 1T, 3T y 7T. Se elige la más cercana y se verifica la tolerancia.
3. ON válido añade punto o raya. Pausa de 3T cierra letra; 7T cierra letra y añade espacio; 1T separa símbolos.
4. Una duración inválida genera advertencia y decisión inválida; no se usa la frase esperada para corregirla.
5. Se retorna texto, Morse, decisiones por tramo, duración y avisos.

Estimar niveles de luz y umbral

lstlisting - Inicio de calibrate (receptor.py, lineas 239-285; Python)

```
# 3. Calibración a 0,5 m

import json
from pathlib import Path

import numpy as np



def calibrate(intensidades, fps: float, roi, fuente="", distancia=0.5) -> dict:
    """Estima el umbral de brillo, T y tolerancia usando la secuencia CAL.

    La calibracion oficial se hace a 0,5 m. La secuencia conocida permite
    comprobar que los niveles de brillo y la estructura temporal sean validos.
    """
    if distancia != 0.5:
        raise ValueError("La calibracion oficial debe realizarse a 0,5 m")

    intensidades = np.asarray(intensidades, dtype=float)
    if len(intensidades) < 10 or not np.all(np.isfinite(intensidades)):
        raise ValueError("Calibracion insuficiente o invalida")

    # Se parte de cuantiles para separar aproximadamente los niveles apagado y encendido.
    nivel_apagado, nivel_encendido = np.quantile(intensidades, [0.1, 0.9])

    # Reasigna cada muestra al grupo mas cercano al umbral y actualiza sus promedios.
    for iteracion in range(30):
        umbral_estimado = (nivel_apagado + nivel_encendido) / 2
        esta_encendido = intensidades > umbral_estimado
        if not esta_encendido.any() or esta_encendido.all():
            raise ValueError("No se distinguen niveles ON/OFF")

        nuevo_nivel_apagado = intensidades[~esta_encendido].mean()
        nuevo_nivel_encendido = intensidades[esta_encendido].mean()
        if (
            abs(nuevo_nivel_apagado - nivel_apagado)
            + abs(nuevo_nivel_encendido - nivel_encendido)
            < 1e-6
        ):
            break
        nivel_apagado, nivel_encendido = nuevo_nivel_apagado, nuevo_nivel_encendido

    if nivel_encendido - nivel_apagado < 10:
        raise ValueError("Contraste inferior a 10 unidades digitales; revisar ROI y montaje")

    umbral = float((nivel_apagado + nivel_encendido) / 2)

```

1. La rutina exige distancia 0,5 m y suficientes muestras finitas.
2. Los cuantiles 10% y 90% estiman inicialmente los grupos apagado y encendido.
3. Hasta 30 iteraciones, las muestras se asignan al nivel más cercano y se recalculan las medias.

4. Si todo queda en un grupo o el contraste es menor que 10 niveles digitales, se rechaza la calibración.
5. El umbral final es theta = (OFF + ON)/2.

Comprobar AN E, T y tolerancia

lstlisting - Validación temporal y salida de calibrate (receptor.py, lineas 286-342; Python)

```
    # Detecta tramos ON/OFF y descarta los margenes apagados del video.
    tramos_observados = runs(intensidades, fps, umbral)
    indices_encendidos = [
        indice for indice, tramo in enumerate(tramos_observados) if tramo["state"]
    ]
    if not indices_encendidos:
        raise ValueError("Calibracion sin pulsos")
    tramos_observados = tramos_observados[
        indices_encendidos[0] : indices_encendidos[-1] + 1
    ]

    # Compara la secuencia recibida con los tramos que debe producir AN E.
    tramos_esperados = segmentos(texto_calibracion, 1.)
    if len(tramos_observados) != len(tramos_esperados) or any(
        tramo_observado["state"] != tramo_esperado["state"]
        for tramo_observado, tramo_esperado in zip(tramos_observados, tramos_esperados)
    ):
        raise ValueError("CAL debe contener AN E completo (5 pulsos); revise tramo/ROI")

    # Cada tramo aporta una estimacion de T; la mediana reduce el efecto de medidas atipicas.
    duracion_unidad = float(
        np.median(
            [
                tramo_observado["duration"] / tramo_esperado["units"]
                for tramo_observado, tramo_esperado in zip(
                    tramos_observados, tramos_esperados
                )
            ]
        )
    )
    desviacion_maxima = max(
        abs(tramo_observado["duration"] / duracion_unidad - tramo_esperado["units"])
        for tramo_observado, tramo_esperado in zip(
            tramos_observados, tramos_esperados
        )
    )
    # La tolerancia incluye una base fija y el error temporal de un cuadro.
    tolerancia = max(0.35, desviacion_maxima + 1 / (fps * duracion_unidad))
    if tolerancia >= 1:
        raise ValueError("Calibracion temporal inconsistente; aumentar T o revisar video")

    return dict(
        schema_version=1,
        distance_m=distancia,
        source=str(fuente),
        calibration_text=texto_calibracion,
        threshold=umbral,
        off_level=float(nivel_apagado),
        on_level=float(nivel_encendido),
        t_seconds=duracion_unidad,
        tolerance_units=tolerancia,
        roi=list(map(int, roi)),
        fs_calibration=fps,
        method="Dos grupos de intensidad; mediana duracion/unidades CAL; tolerancia absoluta en T",
        roi_rule="Rectangulo fijo centrado en fuente; mismo ancho y alto en todas las distancias",
        calibration_runs=tramos_observados,
    )
```

1. runs detecta estados y se eliminan márgenes OFF exteriores.
2. Cantidad y orden se comparan con el patrón AN E. Una secuencia incompleta invalida la calibración.
3. Cada tramo aporta duración / unidades ideales; la mediana estima T y limita el efecto de un valor atípico.
4. La tolerancia es el máximo entre 0,35T y la desviación observada más un cuadro expresado en T. Se rechaza si llega a 1T.

5. Se guardan umbral, T, tolerancia, ROI, FPS y tramos observados.

Guardar y validar el JSON de calibración

lstlisting - Funciones save_config y load_config (receptor.py, lineas 345-383; Python)

```python
def save_config(configuracion: dict, ruta):
    """Guarda la configuracion de calibracion como JSON legible."""
    ruta = Path(ruta)
    ruta.parent.mkdir(parents=True, exist_ok=True)
    ruta.write_text(
        json.dumps(configuracion, ensure_ascii=False, indent=2) + "\n", encoding="utf-8"
    )


def load_config(ruta) -> dict:
    """Carga una calibracion y valida version, parametros y region de interes."""
    configuracion = json.loads(Path(ruta).read_text(encoding="utf-8"))

    if configuracion.get("schema_version") != 1:
        raise ValueError("Version de calibracion no compatible")
    if configuracion.get("distance_m") != 0.5:
        raise ValueError("La configuracion debe provenir de calibracion a 0,5 m")

    for nombre_campo in ["threshold", "t_seconds", "tolerance_units"]:
        if not np.isfinite(float(configuracion[nombre_campo])):
            raise ValueError(f"Parametro no finito: {nombre_campo}")

    if (
        not 0 <= configuracion["threshold"] <= 255
        or configuracion["t_seconds"] <= 0
        or not 0 < configuracion["tolerance_units"] < 1
    ):
        raise ValueError("Umbral, T o tolerancia fuera de rango")

    roi = configuracion["roi"]
    if (
        len(roi) != 4
        or any(int(value) != value for value in roi)
        or any(value < 0 for value in roi[:2])
        or any(value <= 0 for value in roi[2:])
    ):
        raise ValueError("ROI de calibracion invalida")

    return configuracion
```

1. save_config crea la carpeta necesaria y escribe JSON legible en UTF-8.
2. load_config exige versión correcta y calibración a 0,5 m.
3. Umbral, T y tolerancia deben ser finitos y estar dentro de rango.
4. La ROI debe tener cuatro enteros, origen no negativo y ancho/alto positivos.

Abrir video y fijar base temporal

lstlisting - Inicio de extract_video (receptor.py, lineas 385-420; Python)

```python
# 4. Lectura del video y medición de la ROI

from pathlib import Path

import cv2
import numpy as np


def extract_video(ruta, roi=None, inicio=0., fin=None, seleccion_interactiva=False) -> dict:
    """Mide la intensidad media de una ROI fija en cada cuadro seleccionado.

    Devuelve la señal de brillo y los metadatos que necesitan el decodificador
    y las etapas posteriores del procesamiento.
    """
    ruta_video = Path(ruta)
    if not ruta_video.is_file():
        raise ValueError(f"Video inexistente: {ruta_video}")
```

lstlisting - Inicio de extract_video (receptor.py, lineas 385-420; Python)

```python
    if not np.isfinite(inicio) or inicio < 0 or (fin is not None and (not np.isfinite(fin) or fin <= inicio)):
        raise ValueError("Intervalo temporal invalido")

    # OpenCV entrega los cuadros; FPS y cantidad declarada permiten comprobar
    # que la secuencia se pueda interpretar con una base de tiempo conocida.
    captura = cv2.VideoCapture(str(ruta_video))
    try:
        if not captura.isOpened():
            raise ValueError("No se pudo abrir el video; comprobar codec")
        frecuencia_muestreo = float(captura.get(cv2.CAP_PROP_FPS))
        cuadros_declarados = int(captura.get(cv2.CAP_PROP_FRAME_COUNT))
        if not np.isfinite(frecuencia_muestreo) or frecuencia_muestreo <= 0:
            raise ValueError("FPS no disponibles; usar un video con temporizacion documentada")

        intensidades, indices_cuadro, marcas_tiempo = [], [], []
        primer_cuadro_rgb = None
        indice_cuadro = 0

        # Se recorre el video completo para seleccionar el intervalo por índice
```

1. La ruta debe existir y los límites temporales deben ser coherentes.
2. OpenCV abre el archivo y entrega FPS y cantidad declarada de cuadros.
3. Se crean listas para intensidad, índices y tiempos.
4. Cada cuadro tiene tiempo nominal índice / FPS; se consulta también el timestamp del backend para verificar compatibilidad.

Medir una ROI por cuadro

lstlisting - Bucle de lectura y promedio de ROI (receptor.py, lineas 422-465; Python)

```python
        while True:
            lectura_correcta, cuadro = captura.read()
            if not lectura_correcta:
                break

            tiempo_cuadro = indice_cuadro / frecuencia_muestreo
            marca_tiempo = captura.get(cv2.CAP_PROP_POS_MSEC) / 1000.
            indice_seleccionado = indice_cuadro
            indice_cuadro += 1

            if tiempo_cuadro < inicio:
                continue
            if fin is not None and tiempo_cuadro >= fin:
                break

            alto_cuadro, ancho_cuadro = cuadro.shape[:2]
            if primer_cuadro_rgb is None:
                # La selección interactiva ocurre una vez; luego se usa la
                # misma ROI para que la curva mida siempre la misma zona.
                if seleccion_interactiva:
                    ventana_roi = "ROI fuente (ENTER confirma)"
                    roi_seleccionada = cv2.selectROI(ventana_roi, cuadro, False, False)
                    cv2.destroyWindow(ventana_roi)
                    roi = list(map(int, roi_seleccionada))
                if roi is None:
                    raise ValueError("Debe especificar ROI x y ancho alto o --seleccionar-roi")
                if len(roi) != 4 or any(int(valor) != valor for valor in roi):
                    raise ValueError("ROI requiere cuatro coordenadas enteras")
                roi = list(map(int, roi))
                # Se conserva el primer cuadro en RGB para mostrarlo en la UI.
                primer_cuadro_rgb = cv2.cvtColor(cuadro, cv2.COLOR_BGR2RGB)

            x, y, ancho_roi, alto_roi = roi
            if (x < 0 or y < 0 or ancho_roi <= 0 or alto_roi <= 0
                    or x + ancho_roi > ancho_cuadro or y + alto_roi > alto_cuadro):
                raise ValueError("ROI fuera del cuadro o vacia")

            # El promedio en escala de grises reduce cada ROI a una muestra de
            # brillo, que luego se compara con el umbral de calibración.
            cuadro_roi = cuadro[y:y + alto_roi, x:x + ancho_roi]
```

lstlisting - Bucle de lectura y promedio de ROI (receptor.py, lineas 422-465; Python)

```
462             roi_gris = cv2.cvtColor(cuadro_roi, cv2.COLOR_BGR2GRAY)
463             intensidades.append(float(roi_gris.mean()))
464             indices_cuadro.append(indice_seleccionado)
465             marcas_tiempo.append(marca_tiempo)
```

1. Se saltan cuadros previos al inicio y se detiene la lectura al llegar al fin solicitado.
2. En el primer cuadro se aceptan coordenadas manuales o se abre la selección interactiva.
3. Se comprueba que el rectángulo esté dentro del cuadro y tenga dimensiones positivas.
4. La ROI se convierte a gris y se calcula su promedio: una muestra digital x[k].
5. Se usa el mismo rectángulo a lo largo del video y se guardan intensidad, índice y timestamp.

Verificar lectura y liberar el archivo

lstlisting - Validación final y retorno de extract_video (receptor.py, lineas 467-507; Python)

```
467         if not intensidades:
468             raise ValueError("No hay cuadros en el tramo seleccionado")
469 
470         cuadros_esperados = (cuadros_declarados if fin is None
471                              else min(cuadros_declarados, int(np.ceil(fin * frecuencia_muestreo))))
472         if cuadros_esperados > 0 and indice_cuadro < cuadros_esperados:
473             raise ValueError("Lectura incompleta del video; conservar archivo y revisar codec/adquisicion")
474 
475         # Los tiempos del backend permiten detectar si los cuadros están
476         # espaciados de forma distinta a la indicada por los FPS nominales.
477         diferencias_marcas = np.diff(marcas_tiempo)
478         advertencias = []
479         if len(diferencias_marcas) and np.all(diferencias_marcas > 0):
480             if np.any(np.abs(diferencias_marcas - 1 / frecuencia_muestreo)
481                       > max(0.003, 0.15 / frecuencia_muestreo)):
482                 raise ValueError("Temporizacion variable detectada; normalizar a FPS constantes y conservar original")
483             temporizacion = "Timestamps del decodificador compatibles con FPS nominal"
484         else:
485             temporizacion = "FPS nominal; timestamps no verificables por backend"
486             advertencias.append("Verificar FPS constantes de la camara; backend sin timestamps fiables")
487 
488         return dict(
489             intensity=np.asarray(intensidades),
490             time=np.asarray(indices_cuadro) / frecuencia_muestreo,
491             fs=frecuencia_muestreo,
492             roi=roi,
493             first_frame=primer_cuadro_rgb,
494             metadata=dict(
495                 source=str(ruta_video.resolve()),
496                 fps=frecuencia_muestreo,
497                 frames=len(intensidades),
498                 frame_size=[ancho_cuadro, alto_cuadro],
499                 start=inicio,
500                 end=fin,
501                 timing=temporizacion,
502                 warnings=advertencias,
503             ),
504         )
505     finally:
506         # Libera el archivo incluso si falla la lectura o la validación.
507         captura.release()
```

1. Se rechaza un segmento sin cuadros y una lectura que termine antes de lo esperado.
2. Las diferencias de timestamp se comparan con 1/FPS; una variación grande detiene el procesamiento.
3. Si no hay timestamps fiables, se advierte que solo se usaron los FPS nominales.
4. Se devuelven señal, tiempos, FPS, ROI y metadatos. finally libera el video incluso ante errores.

Figuras, espectro y CER: siscom/analisis.py

Dibujar intensidad, umbral y espectros

lstlisting - Figuras de señal, FFT y PSD (analisis.py, lineas 1-37; Python)

```python
"""Gráficos, análisis espectral y error de caracteres."""

# 1. Gráficos de señal y resultados

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np


def signal_figure(tiempo, intensidad, umbral):
    """Muestra el brillo medido en la ROI y el umbral ON/OFF."""
    figura, eje = plt.subplots(figsize=(10, 3.5), constrained_layout=True)
    # La línea horizontal permite ver cuándo la señal supera el umbral.
    eje.plot(tiempo, intensidad, lw=1, label="ROI")
    eje.axhline(umbral, color="tab:red", ls="--", label="Umbral fijo")
    eje.set(xlabel="Tiempo (s)", ylabel="Intensidad digital", title="Señal recibida")
    eje.grid(alpha=.25)
    eje.legend()
    return figura


def spectrum_figure(espectro):
    """Compara la FFT de la señal con su PSD y marca el ancho B95."""
    figura, ejes = plt.subplots(2, 1, figsize=(9, 6), constrained_layout=True)
    # La FFT muestra amplitud por frecuencia; Welch estima la densidad espectral.
    ejes[0].plot(espectro["fft_f"], espectro["amplitude"])
    ejes[0].set(xlabel="Frecuencia (Hz)", ylabel="Amplitud digital", title="FFT sin promedio")
    ejes[1].plot(espectro["psd_f"], espectro["psd"])
    ejes[1].axvline(espectro["b95"], color="tab:red", ls="--",
                    label=f"B95 = {espectro['b95']:.3f} Hz")
    ejes[1].set(xlabel="Frecuencia (Hz)", ylabel="Intensidad digital²/Hz", title="PSD Welch")
    ejes[1].legend()
    for eje in ejes:
        eje.set_xlim(0, espectro["fs"]/2)
        eje.grid(alpha=.25)
    return figura
```

1. Matplotlib usa modo Agg, por lo que guarda figuras sin abrir una ventana.
2. signal_figure grafica brillo de ROI versus tiempo y el umbral fijo.
3. spectrum_figure pone amplitud FFT arriba y PSD abajo; marca B95 en el gráfico PSD.
4. Los gráficos muestran hasta FPS/2, límite del espectro unilateral.

Graficar CER y comparar distancias

lstlisting - Figuras de distancia y comparación (analisis.py, lineas 40-77; Python)

```python
def distance_figure(resultados):
    """Grafica el CER de los ensayos válidos en función de la distancia."""
    figura, eje = plt.subplots(figsize=(7,4), constrained_layout=True)
    # Los ensayos sin CER (por ejemplo, sin adquisición válida) no se dibujan.
    resultados_validos = sorted(
        (resultado for resultado in resultados if resultado.get("cer") is not None),
        key=lambda resultado: resultado["distance_m"],
    )
    distancias = [resultado["distance_m"] for resultado in resultados_validos]
    valores_cer = [resultado["cer"] for resultado in resultados_validos]
    eje.plot(distancias, valores_cer, "o-")
    eje.set(xlabel="Distancia (m)", ylabel="CER", title="Errores por distancia")
    eje.grid(alpha=.25)
    return figura


def comparison_figure(ensayo_cercano, ensayo_lejano, umbral, ventana=(0., 3.)):
    """Compara señales a 0,5 m y 3 m alineadas por su primer pulso."""
    figura, ejes = plt.subplots(2,1, figsize=(9,5), sharey=True, constrained_layout=True)
```

lstlisting - Figuras de distancia y comparación (analisis.py, lineas 40-77; Python)

```python
    comparaciones = zip(ejes, [ensayo_cercano, ensayo_lejano], ["0,5 m", "3,0 m"])
    for eje, ensayo, titulo in comparaciones:
        datos_senal, resultado_decodificacion = ensayo["signal"], ensayo["result"]
        if not resultado_decodificacion["decisions"]:
            raise ValueError("No se puede alinear por primer pulso una señal sin pulsos")
        # Restar el inicio del primer pulso permite comparar el mismo tramo Morse.
        tiempo_transcurrido = (
            np.arange(len(datos_senal["intensity"])) / datos_senal["fs"]
            - resultado_decodificacion["decisions"][0]["start"]
        )
        dentro_ventana = (tiempo_transcurrido >= ventana[0]) & (tiempo_transcurrido <= ventana[1])
        if not dentro_ventana.any():
            raise ValueError("Tramo de comparacion sin muestras")
        eje.plot(tiempo_transcurrido[dentro_ventana], datos_senal["intensity"][dentro_ventana], label=titulo)
        eje.axhline(umbral, color="tab:red", ls="--", label="Umbral fijo")
        eje.set(xlabel="Tiempo desde primer pulso (s)", ylabel="Intensidad digital")
        eje.legend()
        eje.grid(alpha=.25)
    return figura
```

1. distance_figure ordena ensayos válidos por distancia y grafica CER. Un ensayo sin CER no se dibuja como error cero.
2. comparison_figure crea paneles de escala vertical común para 0,5 m y 3 m.
3. A cada señal se le resta el tiempo del primer pulso para alinear el origen; luego se recorta la ventana y se dibuja el umbral.
4. Sin pulsos o muestras en la ventana, la función informa que no puede construir la comparación.

Retirar la media, calcular FFT y estimar PSD

lstlisting - Importaciones, FFT y Welch en analyze (analisis.py, lineas 79-122; Python)

```python
# 2. FFT, PSD y B95

import numpy as np
from scipy.signal import welch


def analyze(intensidad, fps: float) -> dict:
    """Analiza una señal de intensidad muestreada a ``fps`` cuadros por segundo.

    La FFT unilateral muestra la amplitud por frecuencia. La PSD se estima con
    el método de Welch y una ventana Hann. B95 es la frecuencia que acumula el
    95 % del área de la PSD, integrada mediante trapecios.
    """
    muestras = np.asarray(intensidad, dtype=float)
    if (
        muestras.ndim != 1
        or len(muestras) < 4
        or not np.all(np.isfinite(muestras))
        or not np.isfinite(fps)
        or fps <= 0
    ):
        raise ValueError("Se requieren al menos 4 muestras finitas y FPS positivos")

    # Quitamos el nivel medio para que el componente constante no domine el espectro.
    muestras_sin_media = muestras - muestras.mean()
    cantidad_muestras = len(muestras)

    # rfft calcula solo frecuencias no negativas. Duplicamos las amplitudes
    # interiores para representar el espectro unilateral (sin duplicar DC/Nyquist).
    amplitud = np.abs(np.fft.rfft(muestras_sin_media)) / cantidad_muestras
    amplitud[1:-1 if cantidad_muestras % 2 == 0 else None] *= 2
    frecuencias_fft = np.fft.rfftfreq(cantidad_muestras, 1 / fps)

    # Welch promedia periodogramas de segmentos solapados para estimar la PSD.
    longitud_segmento = min(256, cantidad_muestras)
    frecuencias_psd, densidad_potencia = welch(
        muestras_sin_media,
        fs=fps,
        window="hann",
```

lstlisting - Importaciones, FFT y Welch en analyze (analisis.py, lineas 79-122; Python)

```
        nperseg=longitud_segmento,
        noverlap=longitud_segmento // 2,
        detrend=False,
        scaling="density",
    )
```

1. NumPy aporta operaciones vectoriales y scipy.signal.welch estima la PSD.
2. Se comprueban al menos cuatro muestras finitas y FPS positivo; luego se retira la media para separar la componente constante del contenido variable.
3. rfft calcula frecuencias no negativas. El eje usa separación FPS/N; amplitudes interiores se duplican para mostrar el espectro unilateral.
4. Welch estima la PSD con ventana Hann, hasta 256 muestras por segmento y 50% de solapamiento. Es la PSD del brillo observado, no la respuesta del canal.

Integrar PSD y encontrar B95

lstlisting - Área espectral y B95 (analisis.py, lineas 124-156; Python)

```
    # La PSD es una densidad: su área representa la potencia de la señal.
    # Integramos por trapecios y buscamos dónde se alcanza el 95 % del área.
    areas_intervalos = (
        (densidad_potencia[:-1] + densidad_potencia[1:])
        * np.diff(frecuencias_psd)
        / 2
    )
    area_acumulada = np.r_[0.0, np.cumsum(areas_intervalos)]
    area_total = float(area_acumulada[-1])
    ancho_b95 = (
        float(np.interp(0.95 * area_total, area_acumulada, frecuencias_psd))
        if area_total > 0
        else 0.0
    )

    return dict(
        fft_f=frecuencias_fft,
        amplitude=amplitud,
        psd_f=frecuencias_psd,
        psd=densidad_potencia,
        cumulative=area_acumulada,
        b95=ancho_b95,
        psd_area=area_total,
        mean=float(muestras.mean()),
        variance=float(np.mean(muestras_sin_media**2)),
        method="Welch",
        window="hann",
        nperseg=longitud_segmento,
        noverlap=longitud_segmento // 2,
        detrend=False,
        scaling="density",
        fs=fps,
    )
```

1. La regla trapezoidal calcula el área entre frecuencias contiguas y la acumula desde cero.
2. Por interpolación se busca la frecuencia donde el área llega al 95% del total: ese valor es B95.
3. Si el área total es cero, B95 se deja en cero.
4. Se devuelven datos FFT/PSD, área, media, varianza y parámetros Welch.

Construir la matriz de Levenshtein

lstlisting - Matriz de costos para CER (analisis.py, lineas 160-202; Python)

```
def cer(referencia: str, recibido: str) -> dict:
    """Calcula sustituciones, eliminaciones, inserciones y CER.

    La referencia solo se usa aquí, después de decodificar el mensaje, para
    medir qué tan parecido quedó el texto recibido al texto transmitido.
```

lstlisting - Matriz de costos para CER (analisis.py, lineas 160-202; Python)

```
    """
    referencia = referencia.upper()
    recibido = recibido.upper()

    if not referencia:
        raise ValueError("La referencia no puede estar vacia")

    cantidad_referencia = len(referencia)
    cantidad_recibida = len(recibido)

    # La tabla guarda el costo mínimo para transformar prefijos de los textos.
    # La fila y columna iniciales representan transformar desde/hacia vacío.
    tabla = [
        [0] * (cantidad_recibida + 1)
        for _ in range(cantidad_referencia + 1)
    ]

    for indice_referencia in range(cantidad_referencia + 1):
        tabla[indice_referencia][0] = indice_referencia

    for indice_recibido in range(cantidad_recibida + 1):
        tabla[0][indice_recibido] = indice_recibido

    # Cada celda elige el menor costo entre sustituir, eliminar o insertar.
    for indice_referencia in range(1, cantidad_referencia + 1):
        for indice_recibido in range(1, cantidad_recibida + 1):
            costo_sustitucion = int(
                referencia[indice_referencia - 1]
                != recibido[indice_recibido - 1]
            )

            tabla[indice_referencia][indice_recibido] = min(
                tabla[indice_referencia - 1][indice_recibido - 1]
                + costo_sustitucion,
                tabla[indice_referencia - 1][indice_recibido] + 1,
                tabla[indice_referencia][indice_recibido - 1] + 1,
            )

```

1. Se convierten referencia y recepción a mayúsculas; la referencia no puede estar vacía.
2. La primera fila y columna inicializan el costo de transformar desde o hacia una cadena vacía.
3. Para cada par de caracteres se calcula sustitución, eliminación e inserción.
4. Cada celda toma el menor de los tres costos; la última celda da el total mínimo de ediciones.

Contar operaciones y normalizar el CER

lstlisting - Recorrido inverso y resultado CER (analisis.py, lineas 203-246; Python)

```
    # Se recorre la tabla hacia atrás para identificar qué operaciones
    # componen el costo mínimo. El orden de las condiciones fija cómo se
    # cuentan los casos ambiguos cuando hay más de un camino mínimo.
    indice_referencia = cantidad_referencia
    indice_recibido = cantidad_recibida
    conteo = {"S": 0, "D": 0, "I": 0}

    while indice_referencia or indice_recibido:
        if indice_referencia and indice_recibido:
            costo_sustitucion = int(
                referencia[indice_referencia - 1]
                != recibido[indice_recibido - 1]
            )
            if (
                tabla[indice_referencia][indice_recibido]
                == tabla[indice_referencia - 1][indice_recibido - 1]
                + costo_sustitucion
            ):
                conteo["S"] += costo_sustitucion
                indice_referencia -= 1
                indice_recibido -= 1
                continue
```

lstlisting - Recorrido inverso y resultado CER (analisis.py, lineas 203-246; Python)

```
225
226         if (
227             indice_referencia
228             and tabla[indice_referencia][indice_recibido]
229             == tabla[indice_referencia - 1][indice_recibido] + 1
230         ):
231             conteo["D"] += 1
232             indice_referencia -= 1
233         else:
234             conteo["I"] += 1
235             indice_recibido -= 1
236
237     # CER se normaliza por la longitud de la referencia, espacios incluidos.
238     errores = tabla[cantidad_referencia][cantidad_recibida]
239     return {
240         **conteo,
241         "errors": errores,
242         "cer": errores / cantidad_referencia,
243         "exact": referencia == recibido,
244         "reference": referencia,
245         "received": recibido,
246     }
```

1. Se recorre la matriz desde abajo a la derecha siguiendo un camino mínimo.
2. Diagonal cuenta coincidencia o sustitución; movimiento vertical, eliminación; horizontal, inserción.
3. Si hay varios caminos mínimos puede variar el reparto S/D/I, pero no el total ni el CER.
4. El denominador es la longitud de la referencia, espacios incluidos.

Encadenamiento y exportación: siscom/procesar.py

Preparar datos y escribir JSON/CSV

lstlisting - Conversión y escritura de archivos (procesar.py, lineas 1-54; Python)

```
1  """Ejecuta el procesamiento de videos y la evaluación de ensayos."""
2
3  # 1. Procesamiento y exportación de un video
4
5  import csv
6  import hashlib
7  import importlib.metadata
8  import json
9  from pathlib import Path
10 import platform
11
12 import numpy as np
13
14 from .receptor import decode
15 from .analisis import analyze
16 from .analisis import signal_figure, spectrum_figure
17 from .receptor import extract_video
18
19
20 def jsonable(obj):
21     """Convierte valores de NumPy en tipos que el módulo json puede guardar."""
22     if isinstance(obj, np.ndarray):
23         return obj.tolist()
24     if isinstance(obj, np.generic):
25         return obj.item()
26     if isinstance(obj, dict):
27         return {clave: jsonable(valor) for clave, valor in obj.items()}
28     if isinstance(obj, (list, tuple)):
29         return [jsonable(valor) for valor in obj]
30     return obj
31
32
33 def write_json(ruta, datos):
34     """Crea el directorio necesario y escribe un objeto como JSON legible."""
35     ruta = Path(ruta)
```

lstlisting - Conversión y escritura de archivos (procesar.py, lineas 1-54; Python)

```
    ruta.parent.mkdir(parents=True, exist_ok=True)
    ruta.write_text(
        json.dumps(jsonable(datos), ensure_ascii=False, indent=2, allow_nan=False) + "\n",
        encoding="utf-8",
    )


def write_csv(ruta, filas, campos=None):
    """Escribe filas como CSV compatible con Excel y otros lectores comunes."""
    ruta = Path(ruta)
    ruta.parent.mkdir(parents=True, exist_ok=True)
    filas = list(filas)
    if campos is None:
        campos = list(filas[0]) if filas else []
    with ruta.open("w", newline="", encoding="utf-8-sig") as flujo:
        escritor = csv.DictWriter(flujo, fieldnames=campos)
        escritor.writeheader()
        escritor.writerows(filas)

```

1. Los imports conectan módulos de extracción, decodificación y análisis.
2. jsonable transforma arreglos y escalares NumPy para poder guardarlos como JSON.
3. write_json crea carpetas y escribe UTF-8 legible; rechaza NaN para conservar JSON válido.
4. write_csv construye encabezados y añade BOM UTF-8 para lectores de hojas de cálculo.

Procesar una grabación

lstlisting - Función process_video (procesar.py, lineas 56-77; Python)

```
def process_video(ruta_video, configuracion, roi=None, inicio=0., fin=None, salida=None):
    """Extrae la señal, la decodifica y calcula su espectro.

    La configuración aporta el umbral y los tiempos de calibración. La ROI se
    puede desplazar, pero su ancho y alto deben coincidir con los calibrados.
    Si se indica ``salida``, también se exportan los resultados.
    """
    roi = configuracion["roi"] if roi is None else roi
    if list(roi)[2:] != list(configuracion["roi"])[2:]:
        raise ValueError("Mantener ancho y alto de ROI de la calibracion")

    datos_senal = extract_video(ruta_video, roi=roi, inicio=inicio, fin=fin)
    resultado = decode(datos_senal["intensity"], datos_senal["fs"], configuracion)
    resultado["warnings"] += datos_senal["metadata"]["warnings"]
    espectro = analyze(datos_senal["intensity"], datos_senal["fs"])
    datos_procesados = dict(
        signal=datos_senal, result=resultado, spectrum=espectro, config=configuracion
    )

    if salida is not None:
        export_result(datos_procesados, salida)
    return datos_procesados
```

1. Si no se indica ROI, se usa la calibrada. Puede desplazarse, pero conserva ancho y alto.
2. extract_video produce x[k]; decode reconstruye Morse y texto.
3. Los avisos de video se agregan a los del decodificador y analyze calcula el espectro.
4. Se agrupan señal, texto, espectro y calibración. La referencia no entra aquí.

Exportar texto y metadatos

lstlisting - Metadatos y trazabilidad (procesar.py, lineas 80-108; Python)

```
def export_result(datos, salida):
    """Guarda el texto, las decisiones, metadatos, datos numéricos y figuras.

    La referencia del mensaje no forma parte de esta exportación: la CER se
    calcula después, durante la evaluación.
    """
```

lstlisting - Metadatos y trazabilidad (procesar.py, lineas 80-108; Python)

```python
    directorio_salida = Path(salida)
    directorio_salida.mkdir(parents=True, exist_ok=True)
    datos_senal = datos["signal"]
    resultado = datos["result"]
    espectro = datos["spectrum"]

    (directorio_salida / "texto_recibido.txt").write_text(resultado["text"], encoding="utf-8")
    write_json(directorio_salida / "resultado.json", resultado)
    write_json(directorio_salida / "calibracion_utilizada.json", datos["config"])

    metadatos = dict(datos_senal["metadata"])
    metadatos["roi"] = datos_senal["roi"]
    metadatos["python"] = platform.python_version()
    metadatos["versions"] = {
        paquete: importlib.metadata.version(paquete)
        for paquete in ["numpy", "scipy", "opencv-python", "matplotlib"]
    }
    with Path(metadatos["source"]).open("rb") as archivo_video:
        metadatos["sha256_video"] = hashlib.file_digest(archivo_video, "sha256").hexdigest()
    metadatos["decision_times"] = "start/end relativos al primer cuadro del tramo extraido"
    metadatos["decision_video_offset_s"] = float(datos_senal["time"][0])
    write_json(directorio_salida / "metadatos.json", metadatos)

```

1. Se crea la carpeta del ensayo y se guardan texto recibido, decisiones y calibración aplicada.
2. Se conservan FPS, tamaño, ROI y versiones de Python y bibliotecas.
3. SHA-256 identifica exactamente qué video fue procesado.
4. Se registra la relación entre tiempos de decisión y tramo seleccionado del video original.

Guardar señal, decisiones, espectros y figuras

lstlisting - Exportación numérica y figuras (procesar.py, lineas 109-151; Python)

```python
    write_csv(
        directorio_salida / "senal.csv",
        (dict(time=tiempo_s, intensity=nivel)
         for tiempo_s, nivel in zip(datos_senal["time"], datos_senal["intensity"])),
    )
    write_csv(
        directorio_salida / "decisiones.csv",
        resultado["decisions"],
        ["state", "start", "end", "duration", "frames", "units_measured",
         "kind", "valid", "pattern"],
    )
    write_csv(
        directorio_salida / "fft.csv",
        (dict(frequency=frecuencia, amplitude=amplitud)
         for frecuencia, amplitud in zip(espectro["fft_f"], espectro["amplitude"])),
    )
    write_csv(
        directorio_salida / "psd.csv",
        (dict(frequency=frecuencia, psd=potencia, cumulative=area)
         for frecuencia, potencia, area in zip(
             espectro["psd_f"], espectro["psd"], espectro["cumulative"]
         )),
    )
    write_json(
        directorio_salida / "espectro.json",
        {clave: valor for clave, valor in espectro.items() if not isinstance(valor, np.ndarray)},
    )

    from matplotlib import pyplot as plt

    figuras = [
        ("senal", signal_figure(
            datos_senal["time"], datos_senal["intensity"], datos["config"]["threshold"]
        )),
        ("espectro", spectrum_figure(espectro)),
    ]
    for nombre, figura in figuras:
```

lstlisting - Exportación numérica y figuras (procesar.py, lineas 109-151; Python)

```
        if datos["config"].get("synthetic"):
            figura.suptitle("DATOS SINTETICOS - prueba de software")
        figura.savefig(directorio_salida / f"{nombre}.png", dpi=160)
        figura.savefig(directorio_salida / f"{nombre}.pdf")
        plt.close(figura)
    return directorio_salida
```

1. senal.csv contiene tiempo e intensidad; decisiones.csv resume cada racha y su validez.
2. fft.csv y psd.csv guardan las curvas; espectro.json guarda parámetros escalares.
3. Se exportan gráficos de señal y espectro en PNG y PDF, y se cierran para liberar memoria.
4. El texto esperado no se copia a esta carpeta individual; se compara durante evaluación.

Validar el manifiesto de cuatro distancias

lstlisting - Inicio de evaluate_trials y validación (procesar.py, lineas 153-210; Python)

```
# 2. Evaluación de los cuatro ensayos

import json
from pathlib import Path

from matplotlib import pyplot as plt

from .analisis import cer
from .analisis import comparison_figure, distance_figure


def evaluate_trials(manifiesto, configuracion, salida):
    """Procesa los ensayos descritos en un manifiesto y guarda sus resultados.

    Las rutas de video relativas se interpretan desde la carpeta del manifiesto.
    ``configuracion`` es la calibración común a todos los ensayos y ``salida`` es la
    carpeta donde se guardan las tablas, gráficas y exportaciones individuales.
    """
    ruta_manifiesto = Path(manifiesto).resolve()
    especificacion = json.loads(ruta_manifiesto.read_text(encoding="utf-8"))
    ensayos = especificacion["trials"]
    distancias = [float(ensayo["distance_m"]) for ensayo in ensayos]

    if sorted(distancias) != [0.5, 1.0, 2.0, 3.0]:
        raise ValueError("Se requieren exactamente cuatro ensayos: 0,5;1;2;3 m")

    referencia = especificacion.get("reference", "SIS COM")
    if referencia != "SIS COM":
        raise ValueError("El ensayo oficial evalua SIS COM, incluyendo espacio")

    carpeta_salida = Path(salida)
    carpeta_salida.mkdir(parents=True, exist_ok=True)
    filas_resultado = []
    procesados_por_distancia = {}

    for ensayo in ensayos:
        distancia = float(ensayo["distance_m"])
        fila_resultado = dict(
            distance_m=distancia,
            duration_s=None,
            S=None,
            D=None,
            I=None,
            errors=None,
            cer=None,
            text="",
            exact=False,
            status="",
            detail="",
            synthetic=bool(especificacion.get("synthetic", False)),
        )

        if ensayo.get("valid", True) is False:
            fila_resultado.update(
```

lstlisting - Inicio de evaluate_trials y validación (procesar.py, lineas 153-210; Python)

```
                status="sin adquisicion valida",
                detail=ensayo.get("note", "Fallo declarado de grabacion"),
            )
        else:
```

1. Se lee el manifiesto JSON. Las rutas relativas se resuelven desde la carpeta del archivo.
2. Se exigen 0,5, 1, 2 y 3 m; la referencia oficial es SIS COM, incluido su espacio.
3. Para cada distancia se prepara una fila. Un ensayo inválido queda registrado sin CER.
4. Los campos vacíos no significan que el mensaje se haya recibido sin errores.

Procesar cada ensayo y calcular CER al final

lstlisting - Procesamiento de cada ensayo (procesar.py, lineas 211-247; Python)

```
            ruta_video = Path(ensayo["video"])
            if not ruta_video.is_absolute():
                ruta_video = ruta_manifiesto.parent / ruta_video

            try:
                salida_ensayo = carpeta_salida / f"distancia_{distancia:g}m"
                datos_video = process_video(
                    ruta_video,
                    configuracion,
                    roi=ensayo.get("roi"),
                    inicio=ensayo.get("start_s", 0.0),
                    fin=ensayo.get("end_s"),
                    salida=salida_ensayo,
                )

                # La referencia se compara después de recibir el texto;
                # no interviene en process_video ni en el decodificador.
                metricas = cer(referencia, datos_video["result"]["text"])
                fila_resultado.update(
                    duration_s=datos_video["result"]["duration"],
                    text=datos_video["result"]["text"],
                    **{
                        clave: metricas[clave]
                        for clave in ["S", "D", "I", "errors", "cer", "exact"]
                    },
                    status="procesado",
                )
                procesados_por_distancia[distancia] = datos_video
            except (ValueError, OSError) as error:
                fila_resultado.update(
                    status="error de procesamiento",
                    detail=str(error),
                )

        filas_resultado.append(fila_resultado)

    write_csv(carpeta_salida / "tabla_resultados.csv", filas_resultado)
```

1. Cada video se procesa con la misma calibración, usando la ROI y el tramo declarados.
2. Solo después de recibir texto se llama a cer(referencia, texto_recibido).
3. La fila guarda distancia, duración, texto, S/D/I, errores, CER y estado.
4. Los errores de lectura se registran en su fila; no se ocultan como resultados válidos.

Exportar tabla y curva CER-distancia

lstlisting - Resumen de evaluación (procesar.py, lineas 248-264; Python)

```
    write_json(
        carpeta_salida / "evaluacion.json",
        dict(
            synthetic=bool(especificacion.get("synthetic", False)),
            reference=referencia,
            trials=filas_resultado,
```

lstlisting - Resumen de evaluación (procesar.py, lineas 248-264; Python)

```
            manifest=especificacion,
            calibration=configuracion,
        ),
    )

    grafico_cer = distance_figure(filas_resultado)
    if especificacion.get("synthetic"):
        grafico_cer.suptitle("DATOS SINTETICOS - no son mediciones fisicas")
    grafico_cer.savefig(carpeta_salida / "cer_distancia.png", dpi=160)
    grafico_cer.savefig(carpeta_salida / "cer_distancia.pdf")
    plt.close(grafico_cer)
```

1. tabla_resultados.csv reúne las filas; evaluacion.json conserva manifiesto, calibración y resultados.
2. distance_figure grafica solo los CER disponibles.
3. La figura se guarda en PNG y PDF y luego se cierra.

Comparar señales de 0,5 m y 3 m

lstlisting - Comparación de señal cercana y lejana (procesar.py, lineas 266-337; Python)

```
    estado_comparacion = "No hay ambos ensayos procesados"
    if 0.5 in procesados_por_distancia and 3.0 in procesados_por_distancia:
        try:
            grafico_comparacion = comparison_figure(
                procesados_por_distancia[0.5],
                procesados_por_distancia[3.0],
                configuracion["threshold"],
                ventana=especificacion.get("comparison_window_s", [0, 3]),
            )
            if especificacion.get("synthetic"):
                grafico_comparacion.suptitle("DATOS SINTETICOS - prueba de software")
            grafico_comparacion.savefig(
                carpeta_salida / "comparacion_0p5_3m.png", dpi=160
            )
            grafico_comparacion.savefig(carpeta_salida / "comparacion_0p5_3m.pdf")
            plt.close(grafico_comparacion)
            estado_comparacion = "alineado por primer pulso; tramo y escala iguales"
        except ValueError as error:
            # Sin pulso detectable en el video lejano, se conserva la
            # comparación del tramo seleccionado y se informa que no está alineada.
            senal_cercana = procesados_por_distancia[0.5]["signal"]
            senal_lejana = procesados_por_distancia[3.0]["signal"]
            grafico_comparacion, ejes = plt.subplots(
                2, 1, sharey=True, figsize=(9, 5), layout="constrained"
            )

            for eje, senal, etiqueta in zip(
                ejes,
                [senal_cercana, senal_lejana],
                ["0,5m", "3,0m"],
            ):
                tiempo_transcurrido = senal["time"] - senal["time"][0]
                inicio_ventana, fin_ventana = especificacion.get(
                    "comparison_window_s", [0, 3]
                )
                dentro_ventana = (tiempo_transcurrido >= inicio_ventana) & (
                    tiempo_transcurrido <= fin_ventana
                )
                eje.plot(
                    tiempo_transcurrido[dentro_ventana],
                    senal["intensity"][dentro_ventana],
                    label=etiqueta,
                )
                eje.axhline(configuracion["threshold"], ls="--", color="tab:red")
                eje.set(
                    xlabel="Tiempo desde inicio seleccionado (s)",
                    ylabel="Intensidad digital",
                )
                eje.legend()

            grafico_comparacion.suptitle(
```

lstlisting - Comparación de señal cercana y lejana (procesar.py, lineas 266-337; Python)

```
                "Sin alineacion por pulso; revisar tramo manual del ensayo"
            )
            if especificacion.get("synthetic"):
                grafico_comparacion.suptitle(
                    "DATOS SINTETICOS - sin alineacion por pulso"
                )
            grafico_comparacion.savefig(
                carpeta_salida / "comparacion_sin_alineacion.png", dpi=160
            )
            plt.close(grafico_comparacion)
            estado_comparacion = (
                str(error) + "; grafico sin alineacion por pulso"
            )

    write_json(
        carpeta_salida / "comparacion.json",
        dict(
            status=estado_comparacion,
            window_s=especificacion.get("comparison_window_s", [0, 3]),
        ),
    )
```

1. El gráfico se intenta solo si ambos videos fueron procesados.
2. Primero se alinea por el primer pulso detectable y se aplica la misma ventana y umbral.
3. Si falta un pulso para alinear, se dibujan los tramos seleccionados desde su inicio y se indica que no hubo alineación.
4. El respaldo no necesariamente compara el mismo símbolo; el JSON deja anotados el estado y la ventana usados.

Marcar simulaciones y retornar resultados

lstlisting - Etiqueta sintética y retorno (procesar.py, lineas 339-345; Python)

```
    if especificacion.get("synthetic"):
        (carpeta_salida / "SOLO_DATOS_SINTETICOS.txt").write_text(
            "No son resultados experimentales del grupo.\n",
            encoding="utf-8",
        )

    return filas_resultado
```

1. Si el manifiesto es sintético, se crea una nota para no confundirlo con una medición experimental.
2. La función retorna las filas; el programa de comandos las imprime y decide el código de salida.

Recorrido completo de ejecución

1. Cargar el .ino al UNO; abrir el Monitor Serial a 115200 baudios y enviar T 300.
2. A 0,5 m grabar CAL; el Arduino emite AN E y Python guarda la configuración estimada.
3. Grabar cuatro transmisiones SEND SIS COM, una por distancia. Python mide ROI, aplica umbral y decodifica sin conocer la referencia.
4. Calcular FFT, PSD y B95; guardar archivos de señal, decisiones y espectro.
5. Al final, comparar los textos recibidos con SIS COM y calcular CER. La palabra esperada nunca se usa para corregir la recepción.